

ГЛАВА 16

ШАБЛОНЫ И ОБОБЩЕННОЕ ПРОГРАММИРОВАНИЕ

В этой главе...

16.1. Определение шаблона	822
16.2. Дедукция аргумента шаблона	853
16.3. Перегрузка и шаблоны	873
16.4. Шаблоны с переменным количеством аргументов	878
16.5. Специализация шаблона	887
Резюме	894
Термины	895

И объектно-ориентированное, и обобщенное программирование имеют дело с типами, неизвестными на момент написания программы. Различие между ними в том, что объектно-ориентированное программирование имеет дело с типами, которые не известны до времени выполнения, тогда как в обобщенном программировании типы становятся известны только во время компиляции.

Все описанные в части II контейнеры, итераторы и алгоритмы являются хорошими примерами обобщенного программирования. При написании обобщенной программы ее код должен работать способом, независимым от специфических типов. При использовании обобщенного кода ему следует предоставить типы или значения, с которыми будет работать данный конкретный экземпляр кода.

Например, библиотека предоставляет единое, обобщенное определение каждого контейнера, такого как вектор. Это обобщенное определение можно использовать для определения множества разных типов векторов, каждый из которых отличается от других типом хранимых элементов.

Шаблоны (template) — это основа обобщенного программирования. Шаблоны вполне можно использовать (как выше в книге), даже не понимая, как они определяются. В этой главе рассматривается определение собственных шаблонов.

В языке C++ шаблоны являются основой для общего программирования. Шаблон — это проект или формула для создания класса или функции.

При использовании такого обобщенного типа, как `vector`, или такой обобщенной функции, как `find()`, следует предоставить дополнительную информацию, необходимую для трансформации их проекта в конкретный класс или функцию во время компиляции. Использование шаблонов рассматривалось в главе 3, а в этой главе мы изучим их определение.

16.1. Определение шаблона

Предположим, необходимо написать функцию, которая сравнивает два значения и указывает, является ли первое из них меньшим, равным или большим, чем второе. Фактически придется создать несколько таких функций, каждая из которых сможет сравнивать значения определенного типа. На первом этапе можно было бы определить несколько перегруженных функций.

```
// возвращает 0, если значения равны, -1, если v1 меньше, и 1,
// если меньше v2
int compare(const string &v1, const string &v2)
{
    if (v1 < v2) return -1;
    if (v2 < v1) return 1;
    return 0;
}

int compare(const double &v1, const double &v2)
{
    if (v1 < v2) return -1;
    if (v2 < v1) return 1;
    return 0;
}
```

Эти функции почти идентичны и отличаются только типом параметров. Тела у обеих функций одинаковы.

Повторение тела функции для каждого сравниваемого типа не только утомительно, но и повышает вероятность возникновения ошибок. Однако важнее всего то, что в этом случае необходимо *заранее* знать все типы, которые придется сравнивать. Этот подход не работает в случае, когда функцию предполагается использовать для типов, неизвестных на данный момент.



16.1.1. Шаблоны функций

Вместо того чтобы определять новую функцию для каждого типа, мы можем определить *шаблон функции* (function template). Шаблон функции — это проект, по которому можно создать некую версию данной функции, специфическую для заданного типа. Шаблон функции `compare()` может выглядеть так:

```
template <typename T>
int compare(const T &v1, const T &v2)
{
    if (v1 < v2) return -1;
```

```

    if (v2 < v1) return 1;
    return 0;
}

```

Определение шаблона начинается с ключевого слова `template`, за которым следует разделяемый запятыми и заключенный в угловые скобки (`<>`) список параметров шаблона (template parameter list), один или несколько параметров шаблона (template parameter).



Список параметров в определении шаблона не может быть пустым

Список параметров шаблона очень похож на список параметров функции. Список параметров функции задает имена и типы локальных переменных, но оставляет их неинициализированными. Инициализацию параметров во время выполнения обеспечивают аргументы.

Аналогично параметры шаблона представляют типы или значения, используемые при определении класса или функции. При использовании шаблона необходимо (явно или неявно) определить *аргументы шаблона* (template argument), чтобы связать их с соответствующими параметрами шаблона.

Например, рассматриваемая функция `compare()` объявляет единственный параметр типа `T`. В шаблоне `compare` имя `T` можно использовать там, где должно быть название типа данных. Фактический тип `T` будет определен компилятором на основании способа применения функции.

Создание экземпляра шаблона функции

Когда происходит вызов шаблона функции, для вывода типов аргументов шаблона компилятор обычно использует аргументы вызова. Таким образом, когда происходит вызов шаблона `compare`, компилятор использует тип аргументов для определения типа, связанного с параметром шаблона `T`. Рассмотрим следующий вызов:

```
cout << compare(1, 0) << endl; // T - тип int
```

Здесь аргумент имеет тип `int`. Компилятор выведет и использует тип `int` как аргумент шаблона, а также свяжет этот аргумент с параметром `T` шаблона.

При создании экземпляра (instantiation) специфической версии функции компилятор сам использует выведенные параметры шаблона. При этом он подставляет фактические аргументы шаблона вместо соответствующих параметров шаблона. Рассмотрим следующий вызов:

```

// создание экземпляра int compare(const int&, const int&)
cout << compare(1, 0) << endl; // T - тип int
// создание
// экземпляра int compare(const vector<int>&, const vector<int>&)
vector<int> vec1{1, 2, 3}, vec2{4, 5, 6};
cout << compare(vec1, vec2) << endl; // T - тип vector<int>

```

Здесь компилятор создает два экземпляра разных версий функции `compare()`. В первой из них параметр `T` заменен типом `int`.

```
int compare(const int &v1, const int &v2)
{
    if (v1 < v2) return -1;
    if (v2 < v1) return 1;
    return 0;
}
```

Во втором вызове создается версия функции `compare()` с параметром `T`, замененным типом `vector<int>`. Такое создание компилятором функций обычно и называют *созданием экземпляра шаблона*.

Параметры типа шаблона

У функции `compare()` есть один *параметр типа* (type parameter) шаблона. Как правило, параметр типа можно использовать как спецификатор типа таким же образом, как и встроенный спецификатор типа или класса. В частности, параметр типа применим при назначении типа возвращаемого значения или типа параметра функции, а также в объявлениях переменных или приведениях в теле функции:

```
// ok: для возвращаемого значения и параметра используется тот же тип
template <typename T> T foo(T* p)
{
    T tmp = *p; // тип tmp совпадает с типом, на который указывает p
    // ...
    return tmp;
}
```

Каждому параметру типа должно предшествовать ключевое слово `class` или `typename`:

```
// ошибка: U должно предшествовать либо typename, либо class
template <typename T, U> T calc(const T&, const U&);
```

В списке параметров шаблона эти ключевые слова имеют одинаковый смысл и применяются взаимозаменяемо. Оба ключевых слова применимы одновременно:

```
// ok: в списке параметров шаблона нет никакой разницы между ключевыми
// словами typename и class
template <typename T, class U> calc (const T&, const U&);
```

Для обозначения параметра типа шаблона интуитивно понятней использовать ключевое слово `typename`, а не `class`; в конце концов, для фактического типа параметра вполне может быть использован встроенный тип, а не только класс. Кроме того, ключевое слово `typename` более точно указывает на то, что следующее за ним имя принадлежит типу. Однако ключевое слово `typename` было добавлено в язык C++ как часть стандарта C++, поэтому в устаревших программах, вероятнее всего, осталось исключительно ключевое слово `class`.

Параметры значения шаблона

Кроме параметров типа, в определении шаблона могут быть использованы *параметры значения* (nontype parameter). Параметр значения представляет значение, а не тип. При определении параметров значения вместо ключевого слова `class` или `typename` используются имена типов.

При создании экземпляра шаблона такие параметры заменяются значением, предоставленным пользователем или выведенным компилятором. Чтобы компилятор смог создать экземпляр шаблона во время компиляции, эти значения должны быть константными выражениями (см. раздел 2.4.4, стр. 103).

В качестве примера напомним версию функции `compare()`, работающую со строковыми литералами. Такие литералы представляют собой массивы типа `const char`. Поскольку скопировать массив нельзя, определим параметры как ссылки на массив (раздел 6.2.4, стр. 286). Поскольку необходима возможность сравнивать литералы разных длин, снабдим шаблон двумя параметрами значения. Первый параметр шаблона представляет размер первого массива, а второй — размер второго:

```
template<unsigned N, unsigned M>
int compare(const char (&p1)[N], const char (&p2)[M])
{
    return strcmp(p1, p2);
}
```

При вызове следующей версии функции `compare()` компилятор будет использовать размер литералов для создания экземпляра шаблона с размерами, которыми заменяют параметры `N` и `M`:

```
compare("hi", "mom")
```

Не забывайте, что компилятор завершает строковый литерал пустым символом (см. раздел 2.1.3, стр. 70). В результате компилятор создаст такой экземпляр:

```
int compare(const char (&p1)[3], const char (&p2)[4])
```

Параметр значения может быть целочисленным типом, указателем, ссылкой на объект (l-значением) или на тип функции. Аргумент, связанный с целочисленным параметром значения, должен быть константным выражением. У аргументов, привязанных к указателю или ссылочному параметру значения, должна быть статическая продолжительность существования (см. главу 12). Нельзя использовать обычный (нестатический) локальный или динамический объект как аргумент шаблона для параметра значения шаблона в виде ссылки или указателя. Параметр-указатель может быть также создан как `nullptr` или нулевое константное выражение.

Параметр значения шаблона — это константное значение в определении шаблона. Параметр значения применим там, где требуются константные выражения, например, при определении размера массива.



Аргументы шаблона, используемые для параметров значения, должны быть константными выражениями.

Шаблоны функции со спецификаторами `inline` и `constexpr`

Шаблон функции может быть объявлен как `inline` (встраиваемый) или `constexpr`, как и обычная функция. Спецификаторы `inline` и `constexpr` располагаются после списка параметров шаблона, но перед типом возвращаемого значения.

```
// ok: спецификатор inline следует за списком параметров шаблона
template <typename T> inline T min(const T&, const T&);
// ошибка: неправильное размещение спецификатора inline
inline template <typename T> T min(const T&, const T&);
```



Создание кода, независимого от типа

Продemonстрируем два наиболее важных принципа создания обобщенного кода на примере функции `compare()`.

- Параметры функций в шаблоне должны быть ссылками на константу.
- При проверке в теле шаблона следует использовать только оператор сравнения `<`.

Объявление параметров функций ссылками на константы гарантирует возможность применения функции к типам, которые не допускают копирования. Большинство типов, включая встроенные типы, но исключая указатели `unique_ptr` и типы ввода-вывода, а также все использованные ранее библиотечные типы допускают копирование. Но вполне могут встретиться и другие типы, которые не допускают копирования. Сделав параметры ссылками на константы, можно гарантировать применимость таких типов в функции `compare()`. Кроме того, если функция `compare()` будет применена для больших объектов, такая конструкция позволит избежать копирования и сэкономить время при выполнении.

Некоторые читатели могут подумать, что для сравнения было бы целесообразней использовать оба оператора `<` и `>`.

```
// ожидаемое сравнение
if (v1 < v2) return -1;
if (v1 > v2) return 1;
return 0;
```

Однако написание кода, использующего только оператор `<`, снизит требования к типам, которые применимы в функции `compare()`. Эти типы должны поддерживать оператор `<`, но не обязаны поддерживать оператор `>`.

Фактически, если действительно следует обеспечить независимость от типа и переносимость кода, лучше определить свою функцию, используя тип `less` (см. раздел 14.8.2, стр. 727):

```
// версия функции compare(), корректно работающая даже с
// указателями; см. р. 14.8.2 (стр. 727)
template <typename T> int compare(const T &v1, const T &v2)
{
    if (less<T>()(v1, v2)) return -1;
    if (less<T>()(v2, v1)) return 1;
    return 0;
}
```

Проблема первоначальной версии в том, что если пользователь вызовет ее с двумя указателями, не указывающими на тот же массив, то результат выполнения кода будет непредсказуем.



При написании кода шаблонов следует постараться минимизировать количество требований, накладываемых на типы аргументов.



Компиляция шаблона

Когда компилятор встречает определение шаблона, он не создает код. Код создается только при создании специфического экземпляра шаблона. Тот факт, что код создается только при использовании шаблона (а не при его определении), влияет как на организацию исходного кода, так и на способы обнаружения ошибок.

Обычно, когда происходит вызов функции, компилятору достаточно объявления функции. Точно так же при использовании объекта класса должно быть доступно определение класса, но определения функций-членов не обязательны. В результате определения классов и объявления функций имеет смысл размещать в файлах заголовка, а определения обычных функций и функций-членов — в файлах исходного кода.

С шаблонами все не так: для создания экземпляра у компилятора должен быть код, определяющий шаблон функции или функцию-член шаблона класса. В результате, в отличие от обычного кода, заголовки для шаблонов обычно включают определения наравне с объявлениями.



Определения шаблонов функций и функций-членов шаблонов классов обычно помещаются в файлы заголовка.

Ключевая концепция. Шаблоны и заголовки

Шаблоны содержат два вида имен:

- не зависящие от параметров шаблона;
- зависящие от параметров шаблона.

Именно разработчик шаблона гарантирует, что все имена, не зависящие от параметров шаблона, будут видимы на момент использования шаблона. Кроме того, разработчик шаблона должен гарантировать видимость определения шаблона, включая определения членов шаблона класса, на момент создания экземпляра шаблона.

Пользователь шаблона должен обеспечить видимость объявлений всех функций, типов и связанных с ними операторов, используемых при создании экземпляра шаблона.

Выполнение этих требований невозможно без хорошо организованной структуры программы, в которой заголовки используются соответствующим образом. Автор шаблона должен предоставить заголовок, который содержит объявления всех имен, используемых в шаблоне класса или в определениях его членов. Прежде чем создать экземпляр шаблона для определенного типа или использовать член класса, созданного по этому шаблону, пользователь должен подключить заголовок для типа шаблона и заголовок, в котором определен используемый тип.

Ошибки компиляции проявляются, главным образом, во время создания экземпляра

Тот факт, что код не создается до создания экземпляра шаблона, влияет на то, когда проявляются ошибки компиляции в коде шаблона. В процессе создания шаблона есть три этапа, во время которых компилятор может сообщить об ошибке.

Первый — когда компилируется само определение шаблона. На этом этапе компилятор, как правило, не может найти большую часть ошибок. Здесь обнаруживаются в основном синтаксические ошибки, такие как пропущенная точка с запятой или неправильно написанное имя переменной, но не более.

Второй этап обнаружения ошибок — когда компилятор встречает применение шаблона. На данном этапе компилятор также способен проверить немного. Для вызова шаблона функции компилятор обычно проверяя количество и типы аргументов. Он может также проверить совпадение типов двух аргументов. Для шаблона класса компилятор может проверить количество и правильность предоставленных аргументов шаблона, но не более.

Третий этап обнаружения ошибок — момент создания экземпляра. Только теперь обнаруживаются ошибки, связанные с типами. В зависимости от того,

как компилятор осуществляет создание экземпляра, он может сообщить об этих ошибках во время редактирования.

При написании шаблона код не может быть открыто специфическим для типа, но можно сделать некоторые предположения об используемых типах. Например, код первоначальной функции `compare()` подразумевал, что тип аргумента имеет оператор `<`.

```
if (v1 < v2) return -1; // для объектов типа T требуется оператор <
if (v2 < v1) return 1; // для объектов типа T требуется оператор <
return 0;              // возвращает int; не зависит от T
```

Когда компилятор обрабатывает тело этого шаблона, он не может проверить корректность условий в операторах `if`. Если переданные функции `compare()` аргументы имеют оператор `<`, то код сработает прекрасно, но не в противном случае. Например:

```
Sales_data data1, data2;
cout << compare(data1, data2) << endl; // ошибка: у Sales_data нет
                                         // оператора <
```

Этот вызов создает экземпляр функции `compare()` с параметром `T`, замененным классом `Sales_data`. Если условия попытаются использовать оператор `<` для объектов класса `Sales_data`, то окажется, что такого оператора нет. В результате получится экземпляр функции, которая не будет откомпилирована. Такие ошибки, как эта, не могут быть обнаружены, пока компилятор не создаст экземпляр определения функции `compare()` для типа `Sales_data`.



ВНИМАНИЕ

Вызывающая сторона должна гарантировать, что переданные шаблону аргументы поддерживают все используемые им операторы, а также то, что эти операторы будут вести себя правильно в том контексте, в котором шаблон использует их.

Упражнения раздела 16.1.1

Упражнение 16.1. Определите создание экземпляра.

Упражнение 16.2. Напишите и проверьте собственные версии функций `compare()`.

Упражнение 16.3. Вызовите собственную функцию `compare()` для объекта класса `Sales_data` и посмотрите, как ваш компилятор обрабатывает ошибки во время создания экземпляра.

Упражнение 16.4. Напишите шаблон, действующий как библиотечный алгоритм `find()`. Функция будет нуждаться в двух параметрах типа шаблона: один — для представления параметров-итераторов функции и другой — для типа значения. Используйте свою функцию для поиска заданного значения в векторе `vector<int>` и списке `list<string>`.

Упражнение 16.5. Напишите шаблон функции `print()` из раздела 6.2.4 (стр. 286), которая получает ссылку на массив и может обрабатывать массивы любого размера и любого типа элементов.

Упражнение 16.6. Как работают библиотечные функции `begin()` и `end()`, получающие аргумент в виде массива? Определите собственные версии этих функций.

Упражнение 16.7. Напишите шаблон `constexpr`, возвращающий размер заданного массива.

Упражнение 16.8. В разделе “Ключевая концепция” на стр. 156 Упомянулось о том, что программисты C++ привыкли использовать оператор `!=`, а не `<`. Объясните причину этой привычки.



16.1.2. Шаблоны класса

Шаблон класса (class template) — своего рода проект для создания классов. Шаблоны классов отличаются от шаблонов функций, для которых компилятор не может вывести типы параметров шаблона. Вместо этого, как уже демонстрировалось не раз, для использования шаблона класса следует предоставить дополнительную информацию в угловых скобках после имени шаблона (см. раздел 3.3, стр. 142). Эта дополнительная информация — список аргументов шаблона, подставляемых вместо параметров шаблона.

Определение шаблона класса

В качестве примера реализуем шаблонную версию класса `StrBlob` (см. раздел 12.1.1, стр. 582). Присвоим шаблону имя `Blob`, указывающее, что он больше не специфичен только для строк. Как и класс `StrBlob`, этот шаблон будет предоставлять совместный (и проверяемый) доступ к своим членам. В отличие от класса, шаблон применяется к элементам практически любого типа. Подобно библиотечным контейнерам, используя шаблон `Blob`, пользователи должны будут определить тип элемента.

Как и шаблоны функций, шаблоны класса начинаются с ключевого слова `template`, за которым следует список параметров шаблона. В определении шаблона класса (и его членов) используются параметры шаблона как знакоместа типов или значений, которые будут подставлены при использовании шаблона:

```
template <typename T> class Blob {
public:
    typedef T value_type;
    typedef typename std::vector<T>::size_type size_type;
    // конструкторы
    Blob();
    Blob(std::initializer_list<T> il);
```

```

// количество элементов в Blob
size_type size() const { return data->size(); }
bool empty() const { return data->empty(); }
// добавление и удаление элементов
void push_back(const T &t) { data->push_back(t); }
// версия перемещения; см. р. 13.6.3 (стр. 684)
void push_back(T &&t) { data->push_back(std::move(t)); }
void pop_back();
// доступ к элементу
T& back();
T& operator[](size_type i); // определено в разделе 14.5 (стр. 716)
private:
    std::shared_ptr<std::vector<T>> data;
    // выдать сообщение, если data[i] недопустим
    void check(size_type i, const std::string &msg) const;
}

```

У шаблона `Blob` есть один параметр типа `T`. Он используется везде, где ожидается тип элемента, хранимый классом `Blob`. Например, тип возвращаемого значения функции доступа к элементам `Blob` определен как `T&`. Когда пользователь создаст экземпляр шаблона `Blob`, он использует параметр `T` для замены конкретным типом аргумента шаблона.

За исключением списка параметров шаблона и использования `T` вместо `string`, этот класс совпадает с тем, что было определено в разделе 12.1.1 (стр. 582) и модифицировано в разделе 12.1.6 (стр. 604), а также в главах 13 и 14.

Создание экземпляра шаблона класса

Как уже неоднократно упоминалось, при использовании шаблона класса следует предоставить дополнительную информацию. Как можно теперь утверждать, эта дополнительная информация является списком *явных аргументов шаблона* (*explicit template argument*), которые привязаны к параметрам шаблона. Компилятор использует эти аргументы для создания специфического экземпляра класса по шаблону.

Например, чтобы определить тип для шаблона `Blob`, следует предоставить тип элемента:

```

Blob<int> ia; // пустой Blob<int>
Blob<int> ia2 = {0,1,2,3,4}; // Blob<int> с пятью элементами

```

Оба объекта, `ia` и `ia2`, используют ту же специфическую для типа версию шаблона `Blob` (т.е. `Blob<int>`). Из этих определений компилятор создает экземпляр класса, который эквивалентен следующему:

```

template <> class Blob<int> {
    typedef typename std::vector<int>::size_type size_type;
    Blob();
    Blob(std::initializer_list<int> il);
    // ...
    int& operator[](size_type i);

```

```
private:
    std::shared_ptr<std::vector<int>>> data;
    void check(size_type i, const std::string &msg) const;
};
```

Когда компилятор создает экземпляр класса из шаблона Blob, он перепишет его, заменяя каждый экземпляр параметра T заданным аргументом шаблона, которым в данном случае является int.

Компилятор создает разный класс для каждого заданного типа элемента:

```
// эти определения создадут экземпляр двух разных типов Blob
Blob<string> names; // Blob содержащий строки
Blob<double> prices; // другой тип элемента
```

Эти определения привели бы к созданию двух разных экземпляров класса: определение names создает класс Blob, в котором каждое вхождение T заменено на string. Определение prices создает класс Blob, где T заменено на double.



При каждом создании экземпляра шаблона класса получается независимый класс. У типа Blob<string> нет никаких отношений с другим типом класса Blob или специальных прав доступа к его членам.



Ссылки на тип шаблона в пределах шаблона

При чтении кода шаблона класса не следует забывать, что имя шаблона класса не является именем самого класса (см. раздел 3.3 стр. 142). Шаблон класса используется для создания экземпляра класса, при этом всегда используются аргументы шаблона.

Непонятным может показаться то, что код в шаблоне класса вообще не использует имя фактического типа (или значения) как аргумент шаблона. Вместо этого как аргументы шаблона зачастую используются собственные параметры. Например, переменная-член data использует два шаблона, vector и shared_ptr. Каждый раз, когда используется шаблон, следует предоставить аргументы шаблона. В данном случае предоставляемый аргумент шаблона имеет тот же тип, который используется при создании экземпляра шаблона Blob. Следовательно, определение переменной-члена data с использованием параметра типа шаблона Blob свидетельствует о том, что переменная-член data является экземпляром указателя shared_ptr на экземпляр шаблона vector, содержащего объекты типа T.

```
std::shared_ptr<std::vector<T>>> data;
```

При создании экземпляра специфического класса Blob, такого как Blob<string>, переменная-член data будет такой:

```
shared_ptr<vector<string>>>
```

Если создать экземпляр `Blob<int>`, то переменная-член `data` будет такой: `shared_ptr<vector<int>>`, и т.д.

Функции-члены шаблонов класса

Подобно любому классу, функции-члены шаблона класса можно определить как в, так и вне тела класса. Как и у любых других классов, члены, определенные в теле, неявно являются встраиваемыми.

Функция-член шаблона класса сама по себе является обычной функцией. Однако у каждого экземпляра шаблона класса есть собственная версия каждого члена. В результате у функции-члена шаблона класса будут те же параметры шаблона, что и у самого класса. Поэтому функция-член, определенная вне тела шаблона класса, начинается с ключевого слова `template`, сопровождаемого списком параметров шаблона класса.

Как обычно, при определении члена класса вне его тела следует указать, к какому классу он принадлежит. Так же, как обычно, имя созданного из шаблона класса включает его аргументы шаблона. При определении члена аргументы шаблона совпадают с параметрами шаблона. Таким образом, для функции-члена класса `StrBlob`, определенной следующим образом:

тип_возвращаемого_значения `StrBlob::имя_члена(список_парам)`

соответствующий член шаблона `Blob` будет выглядеть так:

```
template <typename T>
тип_возвращаемого_значения Blob<T>::имя_члена(список_парам)
```

Функция `check()` и функции доступа к членам

Начнем с определения функции-члена `check()`, проверяющей предоставленный индекс:

```
template <typename T>
void Blob<T>::check(size_type i, const std::string &msg) const
{
    if (i >= data->size())
        throw std::out_of_range(msg);
}
```

Кроме отличия в имени класса и использовании списка параметров шаблона, эта функция идентична первоначальной функции-члену класса `StrBlob`.

Оператор индексирования и функция `back()` используют параметр шаблона для определения типа возвращаемого значения, но в остальном они неизменны:

```
template <typename T>
T& Blob<T>::back()
{
    check(0, "back on empty Blob");
    return data->back();
}
```

```

}
template <typename T>
T& Blob<T>::operator[](size_type i)
{
    // если i слишком велико, check() передаст сообщение и предотвратит
    // доступ к несуществующему элементу
    check(i, "subscript out of range");
    return (*data)[i];
}

```

В первоначальном классе `StrBlob` эти операторы возвращали тип `string&`. Шабоновая версия возвращает ссылку на любой тип, использованный при создании экземпляра шаблона `Blob`.

Функция `pop_back()` почти идентична оригинальной функции-члену класса `StrBlob`:

```

template <typename T> void Blob<T>::pop_back()
{
    check(0, "pop_back on empty Blob");
    data->pop_back();
}

```

Оператор индексирования и функция-член `back()` перегружены как `const`. Оставим определение этих функций-членов и функции `front()` читателю в качестве самостоятельного упражнения.

Конструкторы `Blob()`

Подобно любым другим функциям-членам, определенным вне шаблона класса, конструктор начинается с объявления параметров шаблона для шаблона класса, членом которого он является:

```

template <typename T>
Blob<T>::Blob(): data(std::make_shared<std::vector<T>>()) { }

```

Здесь функция-член `Blob()` определяется в пределах шаблона `Blob<T>`. Как и стандартный конструктор `StrBlob()` (см. раздел 12.1.1, стр. 582), данный конструктор резервирует пустой вектор и сохраняет указатель на него в переменной `data`. Как уже упоминалось, в качестве аргумента резервируемого шаблона `vector` используется собственный параметр типа класса.

Точно так же конструктор, получающий параметр типа `initializer_list`, использует свой параметр типа `T` как тип элемента для своего параметра типа `initializer_list`:

```

template <typename T>
Blob<T>::Blob(std::initializer_list<T> il):
    data(std::make_shared<std::vector<T>>(il)) { }

```

Подобно стандартному конструктору, этот конструктор резервирует новый вектор. В данном случае этот вектор инициализируется из параметра `il`.

Чтобы использовать этот конструктор, следует передать список инициализации, тип элементов которого совместим с типом элемента `Blob`:

```
Blob<string> articles = {"a", "an", "the"};
```

Параметр этого конструктора имеет тип `initializer_list<string>`. Каждый строковый литерал в списке неявно преобразуется в тип `string`.

Создание функций-членов шаблона класса

По умолчанию экземпляр функции-члена шаблона класса создается, *только* если программа использует эту функцию-член. Рассмотрим следующий код:

```
// создает экземпляр Blob<int> и конструктор initializer_list<int>
Blob<int> squares = {0,1,2,3,4,5,6,7,8,9};
// создает экземпляр Blob<int>::size() const
for (size_t i = 0; i != squares.size(); ++i)
    squares[i] = i*i; // создает экземпляр Blob<int>::operator[](size_t)
```

Этот код создает экземпляр класса `Blob<int>` и трех его функций-членов: `operator[]()`, `size()` и конструктора `initializer_list<int>()`.

Если функция-член не используется, ее экземпляр не создается. Благодаря этому факту можно создавать экземпляры класса, используя типы, которые не отвечают требованиям для некоторых из операций шаблона (см. раздел 9.2, стр. 425).



По умолчанию экземпляр члена шаблона класса создается, только если он используется.

Упрощение использования имени шаблона класса в коде класса

Из правила, согласно которому следует предоставить аргументы шаблона при использовании шаблона класса, есть одно исключение. В области видимости самого шаблона класса имя шаблона можно использовать без аргументов:

```
// BlobPtr передает исключение при попытке доступа к несуществующему
// элементу
template <typename T> class BlobPtr
public:
    BlobPtr(): curr(0) { }
    BlobPtr(Blob<T> &a, size_t sz = 0):
        wptr(a.data), curr(sz) { }
    T& operator*() const
    { auto p = check(curr, "dereference past end");
      return (*p)[curr]; // (*p) - вектор, на который указывает этот
                        // объект
    }
    // инкремент и декремент
    BlobPtr& operator++(); // префиксные операторы
    BlobPtr& operator--();
```

```
private:
    // если проверка успешна, check() возвращает shared_ptr на вектор
    std::shared_ptr<std::vector<T>>
        check(std::size_t, const std::string&) const;
    // хранит weak_ptr, а значит, базовый вектор может быть удален
    std::weak_ptr<std::vector<T>> wptr;
    std::size_t curr; // текущая позиция в пределах массива
};
```

Внимательные читатели, вероятно, обратили внимание на то, что префиксные функции-члены инкремента и декремента шаблона класса BlobPtr возвращают тип BlobPtr&, а не BlobPtr<T>&. В области видимости шаблона класса компилятор рассматривает ссылки на сам шаблон так, как будто были подставлены аргументы шаблона, соответствующие собственным параметрам. Таким образом, этот код эквивалентен следующему:

```
BlobPtr<T>& operator++();
BlobPtr<T>& operator--();
```

Использование имени шаблона класса вне тела шаблона

При определении функций-членов вне тела шаблона класса следует помнить, что код находится не в области видимости класса, пока не встретилось имя класса (см. раздел 7.4, стр. 366):

```
// постфикс: осуществляет инкремент/декремент объекта, но возвращает
// неизменное значение
template <typename T>
BlobPtr<T> BlobPtr<T>::operator++(int)
{
    // никакой проверки здесь не нужно; ее выполнит вызов префиксного
    // инкремента
    BlobPtr ret = *this; // сохранить текущее значение
    ++*this;             // перемещение на один элемент; префиксный ++
                        // проверяет инкремент
    return ret;          // вернуть сохраненное состояние
}
```

Поскольку тип возвращаемого значения присутствует вне области видимости класса, следует указать, что он возвращает экземпляр BlobPtr, созданный с тем же типом, что и класс. В теле функции код находится в пределах класса, поэтому не нужно повторять аргумент шаблона при определении ret. Когда аргументы шаблона не предоставлены, компилятор подразумевает, что используется тот же тип, что и при создании экземпляра функции-члена. Следовательно, определение ret будет эквивалентно следующему:

```
BlobPtr<T> ret = *this;
```



В области видимости шаблона класса можно обращаться к шаблону, не определяя его аргументы.

Шаблоны классов и дружественные отношения

Когда класс объявляет дружественные отношения (см. раздел 7.2.1, стр. 351), класс и его друг могут быть или не быть шаблонами. Шаблон класса, у которого есть друг, не являющийся шаблоном, предоставляет дружественный доступ ко всем экземплярам шаблона. Когда друг сам является шаблоном, предоставляющий дружественные отношения класс контролирует, распространяются ли они на все экземпляры шаблона или только на некий специфический экземпляр.

Дружественные отношения “один к одному”

Наиболее распространенная форма дружественных отношений между шаблоном класса и другим шаблоном (класса или функции) подразумевает дружбу между соответствующими экземплярами класса и его друга. Например, класс `Blob` должен объявить дружественным класс `BlobPtr` и шаблонную версию оператора равенства класса `Blob` (первоначально определенную для класса `StrBlob` в упражнении раздела 14.3.1 (стр. 712)).

Чтобы обратиться к определенному экземпляру шаблона (класса или функции), следует сначала объявить сам шаблон. Объявление шаблона включает список параметров шаблона:

```
// для объявления дружественных отношений в шаблоне Blob нужны
// предварительные объявления
template <typename> class BlobPtr;
template <typename> class Blob; // необходимо для параметров operator==
template <typename T>
    bool operator==(const Blob<T>&, const Blob<T>&);
template <typename T> class Blob {
    // каждый экземпляр Blob предоставляет доступ к версии BlobPtr и
    // оператору равенства экземпляра, созданного с тем же типом
    friend class BlobPtr<T>;
    friend bool operator==<T>
        (const Blob<T>&, const Blob<T>&);
    // другие члены, как в разделе 12.1.1 (стр. 582)
};
```

Начнем с объявления `Blob`, `BlobPtr` и `operator==` шаблонами. Эти объявления необходимы для объявления параметра в функции `operator==` и дружественных объявлений в шаблоне `Blob`.

Объявления дружественными используют параметр шаблона `Blob` как собственный аргумент шаблона. Таким образом, дружба ограничивается этими экземплярами шаблона `BlobPtr` и оператора равенства, которые создаются с тем же типом:

```
Blob<char> ca; // BlobPtr<char> и operator==<char> друзья
Blob<int> ia;  // BlobPtr<int> и operator==<int> друзья
```

Члены класса `BlobPtr<char>` могут обращаться к не открытым членам объекта `sa` (или любого другого объекта класса `Blob<char>`), но объект `sa` не имеет никаких специальных прав доступа к объекту `ia` (или любому другому объекту класса `Blob<int>`) или любому другому экземпляру класса `Blob`.

Общие и специфические дружественные отношения шаблонов

Класс может также сделать дружественным каждый экземпляр шаблона или ограничить дружбу специфическим экземпляром:

```
// предварительное объявление необходимо для объявления дружественных
// отношений со специфическим экземпляром шаблона
template <typename T> class Pal;
class C { // C - обычный, не шаблонный класс
    friend class Pal<C>; // экземпляр Pal создается с классом C как
                        // дружественным
    // все экземпляры Pal2 дружественны C;
    // при предоставлении дружественных отношений всем экземплярам
    // предварительное объявление не обязательно
    template <typename T> friend class Pal2;
};

template <typename T> class C2 { // C2 - сам шаблон класса
    // у каждого экземпляра C2 есть тот же экземпляр Pal, что и у друга
    friend class Pal<T>; // объявление шаблона для Pal должно быть в
                        // области видимости
    // все экземпляры Pal2 - друзья каждого экземпляра C2; необходимо
    // предварительное объявление
    template <typename X> friend class Pal2;
    // Pal3 - не шаблонный класс, являющийся другом каждого экземпляра C2
    friend class Pal3; // предварительное объявление для Pal3
                        // не обязательно
};
```

Чтобы позволить создавать все экземпляры как дружественные, объявление дружественных отношений должно использовать параметры шаблона, которые отличаются от используемых самим классом.



Объявление параметра типа шаблона дружественным

По новому стандарту параметр типа шаблона можно сделать дружественным:

```
template <typename Type> class Bar {
    friend Type; // предоставить доступ к типу, используемому для создания
                // экземпляра Bar
    // ...
};
```

Здесь указано, что, независимо от используемого для создания экземпляра типа, класс `Bar` будет дружественным. Таким образом, для некоего типа под названием `Foo` он был бы другом для `Bar<Foo>`, а тип `Sales_data` — другом для `Bar<Sales_data>` и т.д.

Следует заметить, что хотя другом обычно бывает класс или функция, для класса `Var` вполне допустимо создание экземпляра со встроенным типом. Такие дружественные отношения позволяют создавать экземпляры таких классов, как `Var` со встроенными типами.

Псевдонимы типа шаблона

Экземпляр шаблона класса определяет тип класса, и, подобно любому другому типу класса, для экземпляра класса можно определить псевдоним при помощи ключевого слова `typedef` (см. раздел 2.5.1, стр. 106):

```
typedef Blob<string> StrBlob;
```

Это определение типа позволит выполнить код, написанный в разделе 12.1.1 (стр. 582), используя текущую версию шаблона `Blob`, экземпляр которого создан для типа `string`. Поскольку шаблон не тип, ключевое слово `typedef` к шаблону неприменимо. Таким образом, нет никакого способа определить `typedef` для шаблона `Blob<T>`.



Однако новый стандарт позволяет определять псевдоним типа для шаблона класса:

```
template<typename T> using twin = pair<T, T>;
twin<string> authors; // authors - это pair<string, string>
```

где имя `twin` определено как синоним для пар с одинаковыми типами членов. Пользователям типа `twin` достаточно определить его только однажды.

Псевдоним типа шаблона — это синоним для целого семейства классов:

```
twin<int> win_loss; // win_loss - это pair<int, int>
twin<double> area; // area - это pair<double, double>
```

Как и при использовании шаблона класса, при использовании псевдонима `twin` следует указать, какой именно вид `twin` необходим.

При определении псевдонима типа шаблона можно зафиксировать один или несколько параметров шаблона:

```
template <typename T> using partNo = pair<T, unsigned>;
partNo<string> books; // books - это pair<string, unsigned>
partNo<Vehicle> cars; // cars - это pair<Vehicle, unsigned>
partNo<Student> kids; // kids - это pair<Student, unsigned>
```

Здесь имя `partNo` определено как синоним семейства типов, которые являются парами, вторая переменная-член которого имеет тип `unsigned`. Пользователи `partNo` определяют тип первой переменной-члена пары, но не второй.

Статические члены шаблонов класса

Подобно любому другому классу, шаблон класса способен объявить статические члены (см. раздел 7.6, стр. 387):

```
template <typename T> class Foo {
public:
    static std::size_t count() { return ctr; }
    // другие члены интерфейса
private:
    static std::size_t ctr;
    // другие члены реализации
};
```

где `Foo` — шаблон класса, у которого есть открытая статическая функция-член `count()` и закрытая статическая переменная-член `ctr`. У каждого экземпляра шаблона `Foo` будет собственный экземпляр статических членов. Таким образом, для любого конкретного типа `X` будет по одной переменной `Foo<X>::ctr` и одной функции `Foo<X>::count()`. Все объекты типа `Foo<X>` будут совместно использовать ту же переименовую `ctr` и функцию `count()`. Например:

```
// создает экземпляр статических членов Foo<string>::ctr
// и Foo<string>::count
Foo<string> fs;
// все три объекта совместно используют те же члены Foo<int>::ctr
// и Foo<int>::count
Foo<int> fi, fi2, fi3;
```

Подобно любой другой статической переменной-члену, у каждой статической переменной-члена шаблона класса должно быть только одно определение. Однако для каждого экземпляра шаблона класса будет отдельный объект. В результате статическую переменную-член шаблона определяют таким же образом, как и функции-члены этого шаблона:

```
template <typename T>
size_t Foo<T>::ctr = 0; // определение и инициализация ctr
```

Подобно любым другим членам шаблона класса, начнем с определения списка параметров шаблона, сопровождаемого типом и именем определяемого члена. Как обычно, имя члена включает имя класса, которое включает для класса, созданного из шаблона, его аргументы шаблона. Таким образом, когда класс `Foo` создается как экземпляр для специфического типа аргумента шаблона, для этого класса будет создан отдельный экземпляр переменной `ctr` и инициализирован значением 0.

Подобно статическим членам обычного класса, к статическому члену шаблона класса можно обратиться через объект класса или непосредственно, при помощи оператора области видимости. Конечно, чтобы использовать статический член через класс, следует обратиться к его конкретному экземпляру:

```
Foo<int> fi; // создает экземпляр класса Foo<int>
            // и статической переменной-члена ctr
auto ct = Foo<int>::count(); // создает экземпляр Foo<int>::count()
ct = fi.count(); // использует Foo<int>::count()
```

```
ct = Foo::count();           // ошибка: экземпляр какого именно
                             // шаблона создается?
```

Как и любая другая функция-член, экземпляр статической функции-члена создается только при его использовании в программе.

Упражнения раздела 16.1.2

Упражнение 16.9. Что такое шаблон функции? Что такое шаблон класса?

Упражнение 16.10. Что происходит при создании экземпляра шаблона класса?

Упражнение 16.11. Следующее определение шаблона `List` неправильно. Как его исправить?

```
template <typename elemType> class ListItem;
template <typename elemType> class List {
public:
    List<elemType>();
    List<elemType>(const List<elemType> &);
    List<elemType>& operator=(const List<elemType> &);
    ~List();
    void insert(ListItem *ptr, elemType value);
private:
    ListItem *front, *end;
};
```

Упражнение 16.12. Напишите собственные версии шаблонов `Blob` и `BlobPtr`, включая все константные члены, которые не были представлены в тексте.

Упражнение 16.13. Объясните, какой вид дружественных отношений вы выбрали бы для операторов равенства и сравнения шаблона `BlobPtr`.

Упражнение 16.14. Напишите шаблон класса `Screen`, который использует параметры значения для определения высоты и ширины экрана.

Упражнение 16.15. Реализуйте операторы ввода и вывода для своего шаблона `Screen`. Какие друзья необходимы классу `Screen` (если таковые вообще имеются) для работы операторов ввода и вывода? Объясните, зачем нужно каждое объявление дружественным (если таковые вообще имеются).

Упражнение 16.16. Перепишите класс `StrVec` (см. раздел 13.5, стр. 666), как шаблон `Vec`.



16.1.3. Параметры шаблона

Подобно именам параметров функций, имена параметров шаблона не имеют никакого значения. Обычно параметрам типа присваивают имя `T`, но можно использовать любое другое:

```
template <typename Foo> Foo calc(const Foo& a, const Foo& b)
{
```

```

Foo tmp = a; // тип tmp совпадает с типом параметров и возвращаемого
              // значения
// ...
return tmp; // типы возвращаемого значения и параметров совпадают
}

```

Параметры шаблона и область видимости

Параметры шаблона следуют обычным правилам области видимости. Имя параметра шаблона применимо сразу после его объявления и до конца объявления или определения шаблона. Подобно любым другим именам, параметр шаблона скрывает любые объявления имен во внешней области видимости. Однако, в отличие от большинства других контекстов, имя, используемое как параметр шаблона, не может быть повторно использовано в пределах шаблона:

```

typedef double A;
template <typename A, typename B> void f(A a, B b)
{
    A tmp = a; // tmp имеет тип параметра шаблона A, а не double
    double B; // ошибка: повторное объявление параметра шаблона B
}

```

Согласно обычным правилам сокрытия имен, определение typedef типа A скрывается определением параметра типа по имени A. Таким образом, переменная tmp не будет иметь тип double; она будет иметь любой тип, который будет передан параметру шаблона A при использовании шаблона. Поскольку нельзя многократно использовать имена параметров шаблона, объявление переменной по имени B ошибочно.

Поскольку имя параметра не может быть использовано многократно, в каждом списке параметров шаблона имя параметра шаблона может присутствовать только однажды:

```

// ошибка: повторение имени V в параметрах шаблона недопустимо
template <typename V, typename V> // ...

```

Объявления шаблона

Объявление шаблона должно включить параметры шаблона:

```

// объявляет, но не определяет compare и Blob
template <typename T> int compare(const T&, const T&);
template <typename T> class Blob;

```

Подобно параметрам функций, имена параметров шаблона не должны совпадать с таковыми в объявлениях и определениях того же шаблона:

```

// все три случая использования calc относятся к тому же шаблону функции
template <typename T> T calc(const T&, const T&); // объявление
template <typename U> U calc(const U&, const U&); // объявление
// определение шаблона
template <typename Type>
Type calc(const Type& a, const Type& b) { /* ... */ }

```

Конечно, у каждого объявления и определения шаблона должно быть то же количество и вид (т.е. тип или значение) параметров.



По причинам, рассматриваемым в разделе 16.3 (стр. 877), объявления всех шаблонов, необходимых данному файлу, обычно располагаются вместе в начале файла перед любым использующим их кодом.

Использование членов типа

Помните, как в разделах 7.4 (стр. 366) и 7.6 (стр. 388) использовался оператор области видимости (`::`) для обращения к статическим членам и членам типа. В обычном коде (не шаблона) у компилятора есть доступ к определению класса. В результате он знает, является ли имя, к которому обращаются через оператор области видимости, типом или статическим членом. Например, в коде `string::size_type`, компилятор имеет определение класса `string` и может узнать, что `size_type` — это тип.

С учетом того, что `T` является параметром типа шаблона, когда компилятор встретит такой код, как `T::mem`, он не будет знать до времени создания экземпляра, является ли `mem` типом или статической переменной-членом. Но чтобы обработать шаблон, компилятор должен знать, представляет ли имя тип. Например, если `T` является именем параметра типа, то как компилятор воспримет следующий код:

```
T::size_type * p;
```

Он должен знать, определяется ли переменная по имени `p` или происходит умножение статической переменной-члена по имени `size_type` на переменную по имени `p`.

По умолчанию язык подразумевает, что имя, к которому обращаются через оператор области видимости, не является типом. В результате, если необходимо использовать тип-член параметра типа шаблона, следует явно указать компилятору, что имя является типом. Для этого используется ключевое слово `typename`:

```
template <typename T>
typename T::value_type top(const T& c)
{
    if (!c.empty())
        return c.back();
    else
        return typename T::value_type();
}
```

Функция `top()` ожидает контейнер в качестве аргумента, она использует ключевое слово `typename` для определения своего типа возвращаемого значе-

ния и создает инициализированный по умолчанию элемент (см. раздел 7.5.3, стр. 378), чтобы возвратить его, если у контейнера с нет никаких элементов.



Когда необходимо уведомить компилятор о том, что имя представляет тип, следует использовать ключевое слово `typename`, а не `class`.

Аргументы шаблона по умолчанию



Аналогично тому, как можно предоставить аргументы по умолчанию для параметров функции (см. раздел 6.5.1, стр. 308), можно предоставить *аргументы шаблона по умолчанию* (default template argument). По новому стандарту можно предоставлять аргументы по умолчанию и для шаблонов функций, и для шаблонов классов. Прежние версии языка допускали аргументы по умолчанию только для шаблонов класса.

В качестве примера перепишем функцию сравнения, использующую по умолчанию библиотечный шаблонный объект функции `less` (см. раздел 14.8.2, стр. 726):

```
// compare() имеет аргумент шаблона по умолчанию, less<T>
// и заданный по умолчанию аргумент функции, F()
template <typename T, typename F = less<T>>
int compare(const T &v1, const T &v2, F f = F())
{
    if (f(v1, v2)) return -1;
    if (f(v2, v1)) return 1;
    return 0;
}
```

Здесь в шаблон добавлен второй параметр типа, `F`, представляющий тип вызываемого объекта (см. раздел 10.3.2, стр. 497), и определен новый параметр функции, `f`, который будет связан с вызываемым объектом.

Предоставлено также значение по умолчанию для этого параметра шаблона и соответствующего ему параметра функции. Аргумент шаблона по умолчанию определяет, что функция `compare()` будет использовать библиотечный класс `less` объекта функции, экземпляр которого создается с тем же параметром типа, что и функция `compare()`. Заданный по умолчанию аргумент функции указывает, что параметр `f` будет инициализирован по умолчанию объектом типа `F`.

Когда пользователи вызывают эту версию функции `compare()`, они могут предоставить собственный оператор сравнения, но не обязаны делать это:

```
bool i = compare(0, 42); // использует less; i равно -1
// результат зависит от isbn в item1 и item2
Sales_data item1(cin), item2(cin);
bool j = compare(item1, item2, compareIsbn);
```

Первый вызов использует заданный по умолчанию аргумент функции, которым является объект типа `less<T>`. В этом вызове `T` имеет тип `int`, поэтому у объекта будет тип `less<int>`. Этот экземпляр функции `compare()` будет использовать для сравнения тип `less<int>`.

Во втором вызове передается функция `compareIsbn()` (см. раздел 11.2.2, стр. 543) и два объекта типа `Sales_data`. Когда функция `compare()` вызывается с тремя аргументами, типом третьего аргумента должен быть вызываемый объект, возвращающий тип, приводимый к типу `bool` и получающий аргументы типа, совместимого с типами первых двух аргументов. Как обычно, типы параметров шаблона выводятся из соответствующих им аргументов функции. В этом вызове тип `T` выводится как тип `Sales_data`, а тип `F` — как тип `compareIsbn()`.

Как и с аргументами функций по умолчанию, у параметра шаблона может быть аргумент по умолчанию, только если у всех параметров справа от него также есть аргументы по умолчанию.

Аргументы по умолчанию шаблона и шаблоны класса

Всякий раз, когда используется шаблон класса, за именем шаблона всегда должны следовать угловые скобки. Скобки означают, что класс будет создан как экземпляр шаблона. В частности, если шаблон класса предоставляет аргументы по умолчанию для всех своих параметров и следует использовать именно их, то после имени шаблона следует поместить пустую пару угловых скобок:

```
template <class T = int> class Numbers { // по умолчанию T - это int
public:
    Numbers(T v = 0): val(v) { }
    // различные операции с числами
private:
    T val;
};
Numbers<long double> lots_of_precision;
Numbers<> average_precision; // пустые <> означают тип по умолчанию
```

Здесь создаются два экземпляра шаблона `Numbers`: версия `average_precision` — экземпляр `Numbers` с заменой параметра `T` типом `int`; версия `lots_of_precision` — экземпляр `Numbers` с заменой параметра `T` типом `long double`.

Упражнения раздела 16.1.3

Упражнение 16.17. Каковы (если есть) различия между параметром типа, объявленным с ключевым словом `typename` и ключевым словом `class`? Когда должно использоваться ключевое слово `typename`?

Упражнение 16.18. Объясните каждое из следующих объявлений шаблона функции и укажите, допустимы ли они. Исправьте все найденные ошибки.

- (a) `template <typename T, U, typename V> void f1(T, U, V);`
- (b) `template <typename T> T f2(int &T);`
- (c) `inline template <typename T> T foo(T, unsigned int*);`
- (d) `template <typename T> f4(T, T);`
- (e) `typedef char Ctype;`
`template <typename Ctype> Ctype f5(Ctype a);`

Упражнение 16.19. Напишите функцию, получающую ссылку на контейнер и выводящую его элементы. Используйте переменную `size_type` и функцию-член `size()` контейнера для контроля цикла, вывода элементов.

Упражнение 16.20. Перепишите функцию из предыдущего упражнения так, чтобы использовать для контроля цикла итераторы, возвращаемые функциями `begin()` и `end()`.

16.1.4. Шаблоны-члены

У класса (обычного или шаблона класса) может быть функция-член, которая сама является шаблоном. Такие члены называются *шаблонами-членами* (*member template*). Шаблоны-члены не могут быть виртуальными.

Шаблоны-члены обычных (не шаблонных) классов

В качестве примера обычного класса, у которого есть шаблон-член, определим класс, подобный стандартному типу функции удаления (*deleter*), используемой указателем `unique_ptr` (см. раздел 12.1.5, стр. 601). Как и у стандартной функции удаления, у данного класса будет перегруженный оператор вызова функции (см. раздел 14.8, стр. 722), который, получив указатель, выполняет для него оператор `delete`. В отличие от стандартной функции удаления, новый класс будет также выводить сообщения при каждом запуске. Поскольку создаваемую функцию удаления предстоит использовать с любым типом, сделаем оператор вызова шаблоном:

```
// класс объекта функции, вызывающий оператор delete для указателя
class DebugDelete {
public:
    DebugDelete(std::ostream &s = std::cerr): os(s) { }
    // подобно любым шаблонам функции, тип T выводится компилятором
    template <typename T> void operator()(T *p) const
    { os << "deleting unique_ptr" << std::endl; delete p; }
private:
    std::ostream &os;
};
```

Как и любой другой шаблон, шаблон-член начинается с собственного списка параметров шаблона. У каждого объекта класса `DebugDelete` есть переменная-член типа `ostream` для вывода и функция-член, которая сама является шаблоном. Этот класс можно использовать вместо оператора `delete`:

```
double* p = new double;
DebugDelete d; // объект, способный действовать как оператор delete
d(p); // вызывает DebugDelete::operator()(double*), удаляющий p
int* ip = new int;
// вызывает operator()(int*) для временного объекта DebugDelete
DebugDelete()(ip);
```

Поскольку вызов объекта DebugDelete удаляет переданный ему указатель, его можно также использовать как функцию удаления для указателя unique_ptr. Чтобы переопределить функцию удаления указателя unique_ptr, укажем тип функции удаления в скобках и предоставим объект типа функции удаления конструктору (см. раздел 12.1.5, стр. 601):

```
// удалить объект, на который указывает p
// создает экземпляр DebugDelete::operator()<int>(int *)
unique_ptr<int, DebugDelete> p(new int, DebugDelete());
// удаляет объект, на который указывает sp
// создает экземпляр DebugDelete::operator()<string>(string*)
unique_ptr<string, DebugDelete> sp(new string, DebugDelete());
```

Здесь указано, что у функции удаления p будет тип DebugDelete и что предоставлен безымянный объект этого типа в конструкторе p().

Деструктор класса unique_ptr вызывает оператор вызова типа DebugDelete. Таким образом, при каждом вызове деструктора класса unique_ptr создается также экземпляр оператора вызова класса DebugDelete. Таким образом, определения выше создадут следующие экземпляры:

```
// примеры создания экземпляров шаблонов-членов DebugDelete
void DebugDelete::operator()(int *p) const { delete p; }
void DebugDelete::operator()(string *p) const { delete p; }
```

Шаблоны-члены шаблонов класса

Шаблон-член можно также определить и для шаблона класса. В данном случае у и класса, и у его члена будут собственные, независимые параметры шаблона.

В качестве примера снабдим класс Blob конструктором, который получает два итератора, обозначающих диапазон копируемых элементов. Поскольку желательно обеспечить поддержку итераторов в различных видах последовательностей, сделаем этот конструктор шаблоном:

```
template <typename T> class Blob {
    template <typename It> Blob(It b, It e);
    // ...
};
```

У этого конструктора есть свой собственный параметр типа шаблона, It, который он использует для типа двух параметров функции.

В отличие от обычных функций-членов шаблонов класса, шаблоны-члены являются шаблонами функций. При определении шаблона-члена вне тела

шаблона класса следует предоставить список параметров шаблона для шаблона класса и для шаблона функции. Список параметров для шаблона класса располагается сначала, затем следует список параметров шаблона-члена:

```
template <typename T> // параметр типа для класса
template <typename It> // параметр типа для конструктора
    Blob<T>::Blob(It b, It e):
        data(std::make_shared<std::vector<T>>(b, e)) { }
```

Здесь определяется член шаблона класса, у которого есть один параметр типа шаблона T. Сам член является шаблоном функции, имеющий параметр типа It.

Создание экземпляров и шаблоны-члены

Чтобы создать экземпляр шаблона-члена шаблона класса, следует предоставить аргументы для параметров шаблона и класса, и функции. Как обычно, аргументы для параметров шаблона класса определяются типом объекта, через который происходит вызов шаблона-члена. Так же как обычно, компилятор, как правило, выводит тип аргументов шаблона для собственных параметров шаблона-члена из аргументов, переданных при вызове (см. раздел 16.1.1, стр. 823):

```
int ia[] = {0,1,2,3,4,5,6,7,8,9};
vector<long> vi = {0,1,2,3,4,5,6,7,8,9};
list<const char*> w = {"now", "is", "the", "time"};
// создает экземпляр класса Blob<int>
// и конструктор Blob<int> с двумя параметрами типа int*
Blob<int> a1(begin(ia), end(ia));
// создает экземпляр конструктора Blob<int> с двумя параметрами
// типа vector<long>::iterator
Blob<int> a2(vi.begin(), vi.end());
// создает экземпляр класса Blob<string> и конструктор Blob<string>
// с двумя параметрами типа list<const char*>::iterator
Blob<string> a3(w.begin(), w.end());
```

При определении a1 указывается явно, что компилятор должен создать экземпляр шаблона Blob с параметром типа int. Параметр типа для его собственных параметров конструктора будет выведен из типа результатов вызова функций begin(ia) и end(ia). Этим типом является int*. Таким образом, определение a1 создает следующий экземпляр:

```
Blob<int>::Blob(int*, int*);
```

Определение a2 использует уже готовый экземпляр класса Blob<int> и создает экземпляр конструктора с параметром типа It, замененным на vector<short>::iterator. Определение a3 (явно) создает экземпляр шаблона Blob с собственным параметром шаблона типа string и (неявно) экземпляр конструктора шаблона-члена этого класса с собственным параметром типа list<const char*>.

Упражнения раздела 16.1.4

Упражнение 16.21. Напишите собственную версию типа `DebugDelete`.

Упражнение 16.22. Пересмотрите программы `TextQuery` из раздела 12.3 (стр. 617) так, чтобы указатель-член `shared_ptr` использовал тип `DebugDelete` как свою функцию удаления (см. раздел 12.1.4, стр. 597).

Упражнение 16.23. Предскажите, когда будет выполняться оператор вызова в вашей основной программе запроса. Если предсказание неправильно, убедитесь, что понимаете почему.

Упражнение 16.24. Добавьте в свой шаблон `Blob` конструктор, получающий два итератора.



16.1.5. Контроль создания экземпляра

Тот факт, что экземпляр шаблона создается только при его использовании (см. раздел 16.1.1, стр. 827), означает, что создание того же экземпляра может происходить в нескольких объектных файлах. Когда два или более отдельно откомпилированных файла исходного кода используют тот же шаблон с теми же аргументами шаблона, создание экземпляра этого шаблона осуществляется в каждом из этих файлов.



В больших системах дополнительные затраты на создание экземпляра того же шаблона в нескольких файлах могут оказаться существенными. По новому стандарту можно избежать этих дополнительных затрат за счет *явного создания экземпляра* (*explicit instantiation*). Его форма такова:

```
extern template объявление; // объявление создания экземпляра
template объявление;      // определение создания экземпляра
```

где *объявление* — это объявление класса или функции, в котором все параметры шаблона заменены аргументами шаблона. Например:

```
// объявление и определение создания экземпляра
extern template class Blob<string>; // объявление
template int compare(const int&, const int&); // определение
```

Когда компилятор встретит внешнее (`extern`) объявление шаблона, он не будет создавать код его экземпляра в этом файле. Объявление экземпляра как `extern` является обещанием того, что будет и не внешнее создание экземпляра в другом месте программы. Вполне может быть несколько внешних объявлений для каждого экземпляра, однако по крайней мере одно определение экземпляра должно быть.

Поскольку компилятор автоматически создает экземпляр шаблона при его использовании, объявление `extern` должно располагаться перед любым кодом, который использует этот экземпляр:

```
// Application.cc
// экземпляры этих шаблонов должны быть созданы в другом месте программы
extern template class Blob<string>;
extern template int compare(const int&, const int&);
Blob<string> sa1, sa2; // экземпляр создается в другом месте
// экземпляры Blob<int> и его конструктор initializer_list создаются
// в этом файле
Blob<int> a1 = {0,1,2,3,4,5,6,7,8,9};
Blob<int> a2(a1); // экземпляр конструктора копий создается в этом файле
int i = compare(a1[0], a2[0]); // экземпляр создается в другом месте
```

Файл Application.o будет создавать экземпляры класса Blob<int> наряду с его конструктором initializer_list и конструктором копий. Экземпляры функции compare<int> и класса Blob<string> не будут созданы в этом файле. Определения этих шаблонов должны быть в каком-то другом файле программы:

```
// templateBuild.cc
// файл создания экземпляра должен предоставить обычное определение для
// каждого типа и функции, которые другие файлы объявляют внешними
template int compare(const int&, const int&);
template class Blob<string>; // создает экземпляры всех членов
                             // шаблона класса
```

В отличие от объявления, когда компилятор видит определение экземпляра, он создает код. Таким образом, файл templateBuild.o будет содержать определения функции compare() для экземпляра типа int и класса Blob<string>. При построении приложения следует скомпоновать файл templateBuild.o с файлом Application.o.



ВНИМАНИЕ

Для каждого объявления экземпляра где-нибудь в программе должно быть определение явного создания экземпляра.

Определения экземпляров создают экземпляры всех членов

Определение экземпляра для шаблона класса создает экземпляры *всех* членов этого шаблона, включая встраиваемые функции-члены. Когда компилятор видит определение экземпляра, он не может знать, какие функции-члены использует программа. Следовательно, в отличие от обычного способа создания экземпляра шаблона класса, компилятор создает экземпляры *всех* членов этого класса. Даже если член класса не будет использоваться, его экземпляр будет создан все равно. Следовательно, явное создание экземпляра можно использовать только для таких типов, которые применимы со всеми членами данного шаблона.



Определение экземпляра используется только для таких типов, которые применимы со всеми функциями-членами шаблона класса.

Упражнения раздела 16.1.5

Упражнение 16.25. Объясните значение этих объявлений:

```
extern template class vector<string>;
template class vector<Sales_data>;
```

Упражнение 16.26. Предположим, что класс NoDefault не имеет стандартного конструктора. Можно ли явно создать экземпляр `vector<NoDefault>`? Если нет, то почему?

Упражнение 16.27. Объясните по каждому помеченному оператору, происходит ли создание экземпляра. Если создается экземпляр шаблона, объясните, почему; если нет, то тоже почему.

```
template <typename T> class Stack { };
void f1(Stack<char>);                // (a)
class Exercise {
    Stack<double> &rsd;               // (b)
    Stack<int>    si;                 // (c)
};
int main() {
    Stack<char> *sc;                  // (d)
    f1(*sc);                         // (e)
    int iObj = sizeof(Stack< string >); // (f)
}
```



16.1.6. Эффективность и гибкость

Библиотечные типы интеллектуальных указателей (см. раздел 12.1, стр. 574) являются хорошим примером грамотно спроектированных шаблонов.

Очевидное различие между указателями `shared_ptr` и `unique_ptr` в стратегии, которую они используют для управления содержащимися в них указателями: один класс предоставляет совместную собственность; а другой — единоличную собственность на хранимый указатель. Это различие и является основанием для создания данных классов.

Данные классы отличаются также тем, как они позволяют пользователям переопределять свою стандартную функцию удаления. Для переопределения функции удаления класса `shared_ptr` достаточно предоставить ему при создании вызываемый объект или функцию `reset()`. У объекта класса `unique_ptr`, напротив, тип функции удаления является частью типа. При определении указателя `unique_ptr` пользователи должны предоставлять этот тип как явный аргумент шаблона. В результате для указателя `unique_ptr` сложнее предоставить собственную функцию удаления.

Различие в способе работы функции удаления — это лишь частность функциональных возможностей данных классов. Но, как будет вскоре продемонстрировано, это различие в стратегии реализации может серьезно повлиять на производительность.

Привязка функции удаления во время выполнения

Даже не зная, как именно реализуются библиотечные типы, вполне можно догадаться, что указатель `shared_ptr` обращается к своей функции удаления косвенно. Поэтому функция удаления должна храниться как указатель или как класс (такой как `function` из раздела 14.8.3 (стр. 730)), инкапсулирующий указатель.

То, что тип функции удаления не известен до времени выполнения, позволяет убедиться, что класс `shared_ptr` не содержит функцию удаления как непосредственный член класса. Действительно, класс `shared_ptr` позволяет изменить тип функции удаления на протяжении продолжительности его существования. Вполне можно создать указатель `shared_ptr`, используя функцию удаления одного типа, а впоследствии использовать функцию `reset()`, чтобы использовать для того же указателя `shared_ptr` другой тип функции удаления. Вообще, у класса не может быть члена, тип которого изменяется во время выполнения. Следовательно, функция удаления должна храниться отдельно.

Размышляя о том, как должна работать функция удаления, предположим, что класс `shared_ptr` хранит контролируемый указатель в переменной-члене класса по имени `p`, а обращение к функции удаления осуществляется через член класса по имени `del`. Деструктор класса `shared_ptr` должен включать такой оператор:

```
// значение del станет известно только во время выполнения; вызов
// через указатель
del ? del(p) : delete p; // вызов del(p) требует перехода во время
                        // выполнения к области хранения del
```

Поскольку функция удаления хранится отдельно, вызов `del(p)` требует перехода во время выполнения к области хранения `del` и выполнения кода, на который он указывает.

Привязка функции удаления во время компиляции

Теперь давайте подумаем, как мог бы работать класс `unique_ptr`. В этом классе тип функции удаления является частью типа `unique_ptr`. Таким образом, у шаблона `unique_ptr` есть два параметра шаблона: представляющий контролируемый указатель и представляющий тип функции удаления. Поскольку тип функции удаления является частью типа `unique_ptr`, тип функции-члена удаления известен на момент компиляции. Функция удаления может храниться непосредственно в каждом объекте класса `unique_ptr`.

Деструктор класса `unique_ptr` работает подобно таковому у класса `shared_ptr`, в котором он вызывает предоставленную пользователем функцию удаления или выполняет оператор `delete` для хранимого указателя:

```
// del связывается во время компиляции; создается экземпляр прямого  
// вызова функции удаления  
del(p); // нет дополнительных затрат во время выполнения
```

Тип `del` — это либо заданный по умолчанию тип функции удаления, либо тип, предоставленный пользователем. Это не имеет значения; так или иначе, выполняемый код будет известен во время компиляции. Действительно, если функция удаления похожа на класс `DebugDelete` (см. раздел 16.1.4, стр. 846), этот вызов мог бы даже быть встраиваемым во время компиляции.

При привязке функции удаления во время компиляции класс `unique_ptr` избегает во время выполнения дополнительных затрат на косвенный вызов своей функции удаления. При привязке функции удаления во время выполнения класс `shared_ptr` облегчает пользователю переопределение функции удаления.

Упражнения раздела 16.1.6

Упражнение 16.28. Напишите собственные версии классов `shared_ptr` и `unique_ptr`.

Упражнение 16.29. Пересмотрите свой класс `Blob` так, чтобы использовать собственную версию класса `shared_ptr`, а не библиотечную.

Упражнение 16.30. Повторно выполните некоторые из своих предыдущих программ, чтобы проверить собственные переделанные классы `shared_ptr` и `Blob`. (Примечание: реализация типа `weak_ptr` не рассматривается в этом издании, поэтому не получится использовать класс `BlobPtr` с пересмотренным классом `Blob`.)

Упражнение 16.31. Объясните, как компилятор мог бы встроить вызов функции удаления, если бы с классом `unique_ptr` был использован класс `DebugDelete`.

16.2. Дедукция аргумента шаблона

Как уже упоминалось, для определения параметров шаблона для шаблона функции компилятор по умолчанию использует аргументы в вызове. Процесс определения аргументов шаблона по аргументам функции называется *дедукцией аргумента шаблона* (template argument deduction). В ходе дедукции аргумента шаблона компилятор использует типы аргументов вызова для поиска таких аргументов шаблона, которые обеспечат лучшее соответствие создаваемой версии функции для данного вызова.



16.2.1. Преобразования и параметры типа шаблона

Подобно нешаблонным функциям, передаваемые в вызове шаблона функции аргументы используются для инициализации параметров этой функции. Параметры функции, тип которых использует параметр типа шаблона, имеют специальные правила инициализации. Только очень ограниченное количество автоматических преобразований применимо к таким аргументам. Вместо преобразования аргументов компилятор создает новые экземпляры.

Как обычно, спецификаторы `const` верхнего уровня (см. раздел 2.4.3, стр. 101) в параметре или аргументе игнорируются. Единственными остальными преобразованиями, выполняемыми при вызове шаблона функции, являются следующие.

- Преобразования констант: параметр функции, являющийся ссылкой (или указателем) на константу, может быть передан как ссылка (или указатель) на не константный объект (см. раздел 4.11.2, стр. 222).
- Преобразование массива или функции в указатель: если тип параметра функции не будет ссылочным, то к аргументам типа массива или функции будет применено обычное преобразование указателя. Аргумент типа массива будет преобразован в указатель на его первый элемент. Точно так же аргумент типа функции будет преобразован в указатель на тип функции (см. раздел 4.11.2, стр. 221).

Другие преобразования, такие как арифметические преобразования (см. раздел 4.11.1, стр. 219), преобразования производного в базовый (см. раздел 15.2.2, стр. 754) и пользовательские преобразования (см. разделы 7.5.4, стр. 380 и 14.9, стр. 732) не выполняются.

В качестве примера рассмотрим вызовы функции `fobj()` и `fref()`. Функция `fobj()` копирует свои параметры, тогда как параметры функции `fref()` являются ссылками:

```
template <typename T> T fobj(T, T); // аргументы копируются
template <typename T> T fref(const T&, const T&); // ссылки
string s1("a value");
const string s2("another value");
fobj(s1, s2); // вызов fobj(string, string); const игнорируется
fref(s1, s2); // вызов fref(const string&, const string&) использует
               // допустимое преобразования в константу для s1
int a[10], b[42];
fobj(a, b);    // вызов f(int*, int*)
fref(a, b);    // ошибка: типы массивов не совпадают
```

В первой паре вызовов как аргументы передаются строка и константная строка. Даже при том, что эти типы не соответствуют точно друг другу, оба вызова допустимы. В вызове функции `fobj()` аргументы копируются, поэтому не имеет значения, был ли первоначальный объект константой. В вызове функции `fref()` тип параметра — ссылка на константу. Преобразование

в константу для ссылочного параметра является разрешенным преобразованием, поэтому данный вызов допустим.

В следующей паре вызовов как аргументы передаются массивы, отличающиеся размером, а следовательно, имеющие разные типы. В вызове функции `fobj()` различие типов массивов не имеет значения. Оба массива преобразуются в указатели. Типом параметра шаблона в функции `fobj` является `int*`. Вызов функции `fref()`, однако, недопустим. Когда параметр является ссылкой, массивы не преобразовываются в указатели (см. раздел 6.2.4, стр. 286). Типы `a` и `b` не совпадают, поэтому вызов ошибочен.



Единственными допустимыми автоматическими преобразованиями для аргументов в параметры типа шаблонов являются преобразование константы в массив или функций в указатель.

Параметры функций с одинаковым типом параметра шаблона

Параметр типа шаблона применим как тип нескольких параметров функции. Поскольку набор преобразований ограничен, аргументы таких параметров должны быть, по существу, того же типа. Если выведенные типы не совпадают, то вызов ошибочен. Например, функция `compare()` (см. раздел 16.1.1, стр. 822) получает два параметра `const T&`. У ее аргументов должен быть фактически тот же тип:

```
long lng;
compare(lng, 1024); // ошибка: нельзя создать
                    // экземпляр compare(long, int)
```

Этот вызов ошибочен потому, что у аргументов функции `compare()` не совпадают типы. Для первого аргумента выведен аргумент шаблона типа `long`; а для второго — `int`. Эти типы не совпадают, поэтому дедукция аргумента шаблона терпит неудачу.

Если необходимо обеспечить обычные преобразования аргументов, можно определить функцию с двумя параметрами типа:

```
// типы аргумента могут отличаться, но должны быть совместимы
template <typename A, typename B>
int flexibleCompare(const A& v1, const B& v2)
{
    if (v1 < v2) return -1;
    if (v2 < v1) return 1;
    return 0;
}
```

Теперь пользователь может предоставлять аргументы разных типов:

```
long lng;
flexibleCompare(lng, 1024); // ok: вызов flexibleCompare(long, int)
```

Конечно, должен существовать оператор `<`, способный сравнивать значения этих типов.

Обычные преобразования применимы к обычным аргументам

У шаблона функции могут быть параметры, определенные с использованием обычных типов, т.е. типов, которые не задействуют параметр типа шаблона. Такие аргументы не обрабатываются специальным образом; они преобразуются, как обычно, в соответствующий тип параметра (см. раздел 6.1, стр. 269). Рассмотрим, например, следующий шаблон:

```
template <typename T> ostream &print(ostream &os, const T &obj)
{
    return os << obj;
}
```

Тип первого параметра функции известен: `ostream&`. У второго параметра, `obj`, тип параметра шаблона. Поскольку тип параметра `os` фиксирован, при вызове функции `print()` к переданным ему аргументам применимы обычные преобразования:

```
print(cout, 42); // создает экземпляр print(ostream&, int)
ofstream f("output");
print(f, 10); // использует print(ostream&, int);
               // преобразует f в ostream&
```

В первом вызове тип первого аргумента точно соответствует типу первого параметра. Этот вызов задействует ту версию функции `print()`, которая получает тип `ostream&` и тип `int` для создания экземпляра. Во втором вызове первый аргумент имеет тип `ofstream`, а преобразование из `ofstream` в `ostream&` допустимо (см. раздел 8.2.1, стр. 408). Поскольку тип этого параметра не зависит от параметра шаблона, компилятор неявно преобразует `f` в `ostream&`.



Обычные преобразования применимы к аргументам, тип которых не является параметром шаблона.

Упражнения раздела 16.2.1

Упражнение 16.32. Что происходит при дедукции аргумента шаблона?

Упражнение 16.33. Назовите два преобразования типов, допустимых для аргументов функций, при дедукции аргумента шаблона.

Упражнение 16.34. С учетом только следующего кода объясните, допустим ли каждый из этих вызовов. Если да, то каков тип `T`? Если нет, то почему?

```
template <class T> int compare(const T&, const T&);
(a) compare("hi", "world");      (b) compare("bye", "dad");
```

Упражнение 16.35. Какой из следующих вызовов ошибочен (если он есть)? Каков тип `T` допустимых вызовов? В чем проблема недопустимых вызовов?

```
template <typename T> T calc(T, int);
template <typename T> T fcn(T, T);
double d;    float f;    char c;
(a) calc(c, 'c');    (b) calc(d, f);
(c) fcn(c, 'c');    (d) fcn(d, f);
```

Упражнение 16.36. Что происходит при следующих вызовах:

```
template <typename T> f1(T, T);
template <typename T1, typename T2> f2(T1, T2);
int i = 0, j = 42, *p1 = &i, *p2 = &j;
const int *cp1 = &i, *cp2 = &j;
(a) f1(p1, p2);    (b) f2(p1, p2);    (c) f1(cp1, cp2);
(d) f2(cp1, cp2);    (e) f1(p1, cp1);    (e) f2(p1, cp1);
```



16.2.2. Явные аргументы шаблона функции

В некоторых редких случаях компилятор не может вывести типы аргументов шаблона. В других случаях следует позволить пользователю контролировать создание экземпляра шаблона. Оба эти случая наиболее вероятны тогда, когда тип возвращаемого значения функции отличается от типов используемых ею параметров.

Определение явного аргумента шаблона

В качестве примера случая, когда необходимо позволить пользователю задавать тип, определим шаблон функции `sum()`, получающий аргументы двух разных типов. Тип результата будет определять пользователь. Таким образом, пользователь сможет выбрать необходимую ему точность.

Чтобы предоставить пользователю контроль над типом возвращаемого значения, определим третий параметр шаблона, представляющий тип возвращаемого значения:

```
// тип T1 не может быть выведен: он отсутствует в списке параметров
// функции
template <typename T1, typename T2, typename T3>
T1 sum(T2, T3);
```

В данном случае нет никакого аргумента, тип которого мог бы использоваться для вывода типа `T1`. Для этого параметра при каждом вызове функции `sum()` вызывающая сторона должна предоставить *явный аргумент шаблона* (*explicit template argument*).

Явный аргумент шаблона предоставляется вызову тем же способом, что и экземпляру шаблона класса. Явные аргументы шаблона определяются в угловых скобках после имени функции и перед списком аргументов:

```
// T1 определяется явно; T2 и T3 выводятся из типов аргумента
auto val3 = sum<long long>(i, lng); // long long sum(int, long)
```

Этот вызов явно определяет тип параметра T1. Компилятор выведет типы для параметров T2 и T3 из типов переменных i и lng.

Явные аргументы шаблона отвечают соответствующим параметрам шаблона слева направо; первый аргумент шаблона отвечает первому параметру шаблона, второй аргумент — второму параметру и т.д. Явный аргумент шаблона может быть пропущен только для замыкающих (крайних справа) параметров, и то, только если они могут быть выведены из параметров функции. Если функция `sum()` была написана следующим образом:

```
// плохой проект: пользователи вынуждены явно определять все три
// параметра шаблона
template <typename T1, typename T2, typename T3>
T3 alternative_sum(T2, T1);
```

то пользователям придется всегда определять аргументы для всех трех параметров:

```
// ошибка: нельзя вывести начальные параметры шаблона
auto val3 = alternative_sum<long long>(i, lng);
// ok: все три параметра определяются явно
auto val2 = alternative_sum<long long, int, long>(i, lng);
```

Нормальные преобразования применимы к аргументам, определенным явно

По тем же причинам, по которым нормальные преобразования разрешены для параметров, определенных с использованием обычных типов (см. раздел 16.2.1, стр. 856), нормальные преобразования применимы также для аргументов, параметры типа шаблона которых определяются явно:

```
long lng;
compare(lng, 1024);           // ошибка: параметры шаблона не совпадают
compare<long>(lng, 1024);     // ok: создает экземпляр compare(long, long)
compare<int>(lng, 1024);      // ok: создает экземпляр compare(int, int)
```

Как уже упоминалось, первый вызов ошибочен, поскольку у аргументов функции `compare()` должен быть одинаковый тип. Если тип параметра шаблона определен явно, обычные преобразования вполне применимы. Таким образом, вызов `compare<long>()` эквивалентен вызову функции, получающей два параметра `const long&`. Параметр типа `int` автоматически преобразуется в тип `long`. Во втором вызове параметр `T` явно определяется как тип `int`, таким образом, тип аргумента `lng` преобразовывается в `int`.

Упражнения раздела 16.2.2

Упражнение 16.37. Библиотечная функция `max()` имеет два параметра функции и возвращает больший из своих аргументов. У этой функции есть один параметр типа шаблона. Можно ли вызвать функцию `max()`, передав ей аргументы типа `int` и `double`? Если да, то как? Если нет, то почему?

Упражнение 16.38. Когда происходит вызов функции `make_shared()` (см. раздел 12.1.1, стр. 576), следует предоставить явный аргумент шаблона. Объясните, почему этот аргумент необходим и как он используется.

Упражнение 16.39. Используйте явный аргумент шаблона, чтобы сделать возможной передачу двух строковых литералов первоначальной версии функции `compare()` из раздела 16.1.1 (стр. 822).



16.2.3. Замыкающие типы возвращаемого значения и трансформация типа

Применение явного аргумента шаблона для представления типа возвращаемого значения шаблона функции хорошо работает тогда, когда необходимо позволить пользователю определять тип возвращаемого значения. В других случаях обязательное предоставление явного аргумента шаблона налагает дополнительное бремя на пользователя без всяких преимуществ. Например, можно написать функцию, которая получает два обозначающих последовательность итератора и возвращает ссылку на элемент этой последовательности:

```
template <typename It>
??? &fcn(It beg, It end)
{
    // обработка диапазона
    return *beg; // вернуть ссылку на элемент из диапазона
}
```

Точный тип, подлежащий возвращению, неизвестен, но известно, что он будет ссылкой на тип элемента обрабатываемой последовательности:

```
vector<int> vi = {1,2,3,4,5};
Blob<string> ca = { "hi", "bye" };
auto &i = fcn(vi.begin(), vi.end()); // fcn() должна вернуть int&
auto &s = fcn(ca.begin(), ca.end()); // fcn() должна вернуть string&
```



Здесь известно, что функция возвратит ссылку `*beg`, а также, что можно использовать выражение `decltype(*beg)` для получения типа этого выражения. Однако параметр `beg` не существует, пока не встретится список параметров. Чтобы определить эту функцию, следует использовать замыкающий тип возвращаемого значения (см. раздел 6.3.3, стр. 301). Поскольку замыкающий тип располагается после списка параметров, он может использовать параметры функции:

```
// замыкающий тип позволяет объявлять тип возвращаемого значения уже
// после списка параметров
template <typename It>
auto fcn(It beg, It end) -> decltype(*beg)
{
    // обработка диапазона
    return *beg; // вернуть ссылку на элемент из диапазона
}
```

Здесь компилятору было указано, что тип возвращаемого значения функции `fcn()` совпадает с типом, возвращенным при обращении к значению параметра `beg`. Оператор обращения к значению возвращает l-значение (см. раздел 4.1.1, стр. 190), таким образом, выведенный выражением `decltype` тип является ссылкой на тип элемента, обозначаемого параметром `beg`. Следовательно, если функция `fcn()` будет вызвана для последовательности строк, то типом возвращаемого значения будет `string&`. Если это будет последовательность элементов типа `int`, то возвращен будет тип `int&`.

Трансформация типа классов библиотечных шаблонов

Иногда прямого доступа к необходимому типу нет. Например, может возникнуть необходимость в функции, подобной `fcn()`, которая возвращает элемент по значению (см. раздел 6.3.2, стр. 294), а не по ссылке.

Проблема написания этой функции в том, что о передаваемых типах неизвестно почти ничего. Единственные известные в этой функции операции, которые можно использовать, — это операции с итераторами, и нет никаких операций с итераторами, которые возвращают элементы (в противоположность ссылкам на элементы).

Чтобы получить тип элемента, можно использовать библиотечный шаблон *трансформации типа* (`type transformation`). Эти шаблоны определяются в заголовке `type_traits`. Обычно классы заголовка `type_traits` используются для так называемого шаблонного метапрограммирования, не рассматриваемого в данной книге. Однако шаблоны трансформации типа полезны и в обычном программировании. Они описаны в табл. 16.1, а их реализация рассматривается в разделе 16.5 (стр. 892).

В данном случае для получения типа элемента можно использовать шаблон `remove_reference`. У шаблона `remove_reference` один параметр типа шаблона и (открытый) тип-член `type`. Если экземпляр шаблона `remove_reference` создается со ссылочным типом, то тип `type` будет ссылочным. Например, если создать экземпляр `remove_reference<int&>`, то типом `type` будет `int`. Точно так же, если создать экземпляр `remove_reference<string&>`, то типом `type` будет `string` и т.д. Таким образом, при условии, что `beg` — итератор, следующее выражение возвратит тип элемента, на который указывает итератор `beg`:

```
remove_reference<decltype(*beg)>::type
```

Выражение `decltype(*beg)` возвратит ссылочный тип элемента `type`. Выражение `remove_reference::type` удаляет ссылку, оставляя тип самого элемента.

Таблица 16.1. Стандартные шаблоны трансформации типа

Для <i>Mod</i> <T>, где <i>Mod</i> есть	Если T есть	То <i>Mod</i> <T>::type есть
remove_reference	X& или X&& в противном случае	X T
add_const	X&, const X или функция в противном случае	T const T
add_l-value_reference	X& X&& в противном случае	T X& T&
add_r-value_reference	X& или X&& в противном случае	T T&&
remove_pointer	X* в противном случае	X T
add_pointer	X& или X&& в противном случае	X* T*
make_signed	Unsigned X в противном случае	X T
make_unsigned	знаковый тип в противном случае	unsigned T T
remove_extent	X[n] в противном случае	X T
remove_all_extents	X[n1][n2]... в противном случае	X T

Используя шаблон `remove_reference` и замыкающий тип с выражением `decltype`, можно написать собственную функцию, возвращающую копию значения элемента:

```
// для использования типа-члена параметра шаблона следует
// использовать typename; см. р. 16.1.3 (стр. 844)
template <typename It>
auto fcn2(It beg, It end) ->
    typename remove_reference<decltype(*beg)>::type
{
    // обработка диапазона
    return *beg; // вернуть копию элемента из диапазона
}
```

Обратите внимание, что тип-член `type` зависит от параметра шаблона. Таким образом, чтобы указать компилятору, что `type` представляет тип (см. раздел 16.1.3, стр. 844), в объявлении типа возвращаемого значения следует использовать ключевое слово `typename`.

Каждый из описанных в табл. 16.1 шаблонов трансформации типа работает так же, как шаблон `remove_reference`. У каждого шаблона есть открытый член `type`, представляющий тип. Этот тип может быть связан с собственным параметром типа шаблона способом, о котором свидетельствует имя шаблона. Если невозможно (или ненужно) преобразовать параметр шаблона, тип-член `type` имеет тип параметра самого шаблона. Например, если `T` — это тип указателя, то `remove_pointer<T>::type` возвращает тип, на который указывает указатель `T`. Если `T` не указатель, то никакого преобразования не нужно. В данном случае у типа `type` тот же тип, что и у `T`.

Упражнения раздела 16.2.3

Упражнение 16.40. Корректна ли следующая функция? Если нет, то почему? Если она допустима, то каковы ограничения на типы ее аргументов (если они есть) и каков тип возвращаемого значения?

```
template <typename It>
auto fcn3(It beg, It end) -> decltype(*beg + 0)
{
    // обработка диапазона
    return *beg; // вернуть копию элемента из диапазона
}
```

Упражнение 16.41. Напишите версию функции `sum()` с типом возвращаемого значения, который будет гарантированно большим, чтобы содержать результат сложения.



16.2.4. Указатели на функцию и дедукция аргумента

При инициализации или присвоении указателя на функцию (см. раздел 6.7 стр. 322) из шаблона функции для вывода аргументов шаблона компилятор использует тип указателя.

Предположим, например, что есть указатель на функцию, которая возвращает тип `int` и получает два параметра, каждый из которых является ссылкой на `const int`. Этот указатель можно использовать для указания на экземпляр функции `compare()`:

```
template <typename T> int compare(const T&, const T&);
// pfl указывает на экземпляр int compare(const int&, const int&)
int (*pfl)(const int&, const int&) = compare;
```

Тип параметров `pfl` определяет тип аргумента шаблона для параметра `T`. Аргументом шаблона для параметра `T` будет `int`. Указатель `pfl` указывает на экземпляр функции `compare()` с параметром `T`, связанным с типом `int`. Если аргументы шаблона не могут быть выведены из типа указателя функции, произойдет ошибка:

```
// перегруженные версии func(); каждая получает разный тип указателя
// функции
void func(int(*) (const string&, const string&));
void func(int(*) (const int&, const int&));
func(compare); // ошибка: какой из экземпляров compare?
```

Проблема в том, что, глядя на тип параметра функции `func()`, невозможно определить уникальный тип для аргумента шаблона. Вызов функции `func()` мог бы создать экземпляр версии функции `compare()`, получающей целые числа или версию, получающую строки. Поскольку невозможно идентифицировать уникальный экземпляр для аргумента функции `func()`, этот вызов не будет откомпилирован.

Неоднозначность вызова функции `func()` можно устранить при помощи явных аргументов шаблона:

```
// ok: явно определенная версия экземпляра compare()
func(compare<int>); // передача compare(const int&, const int&)
```

Это выражение вызывает версию функции `func()`, получающую указатель на функцию с двумя параметрами типа `const int&`.



Когда возвращается адрес экземпляра шаблона функции, контекст должен позволять однозначно идентифицировать тип или значение для каждого параметра шаблона.



16.2.5. Дедукция аргумента шаблона и ссылки

Чтобы лучше понять дедукцию типа, рассмотрим такой вызов функции, где параметр функции `p` является ссылкой на параметр типа шаблона `T`:

```
template <typename T> void f(T &p);
```

Обратите внимание на два момента: здесь применяются обычные правила привязки ссылок; и спецификаторы `const` здесь нижнего уровня, а не верхнего.

Дедукция типа из параметров ссылки на l-значения функций

Когда параметр функции представляет собой обычную ссылку (l-значение) на параметр типа шаблона (т.е. имеющего форму `T&`), правила привязки гласят, что передавать можно только l-значения (например, переменная или выражение, возвращающее ссылочный тип). Этот аргумент может быть или не быть константным. Если аргумент будет константой, то тип `T` будет выведен как константный:

```
template <typename T> void f1(T&); // аргумент должен быть l-значением
// вызовы f1() используют ссылочный тип аргумента как тип параметра
// шаблона
f1(i); // i - это int; параметр шаблона T - это int
f1(ci); // ci - это const int; параметр шаблона T - это const int
f1(5); // ошибка: аргумент ссылочного параметра должен быть l-значением
```

Если параметр функции имеет тип `const T&`, обычные правила привязки гласят, что можно передать любой вид аргумента — объект (константный или нет), временный объект или литеральное значение. Когда сам параметр функции является константой, выведенный для параметра `T` тип не будет константным типом. Константность является частью типа параметра функции, и поэтому она не становится также частью типа параметра шаблона:

```
template <typename T> void f2(const T&); // может получать r-значения
// параметр в f2() - это const &; const в аргументе неуместен
// в каждом из этих трех вызовов параметр функции f2() выводится
// как const int&
f2(i); // i - это int; параметр шаблона T - это int
f2(ci); // ci - это const int, но параметр шаблона T - это int
f2(5); // параметр const & может быть привязан к r-значению; T - это int
```

Дедукция типа из параметров ссылки на r-значения функций

Когда параметр функции является ссылкой на r-значение (см. раздел 13.6.1, стр. 673), т.е. имеет форму `T&&`, обычные правила привязки гласят, что этому параметру можно передать r-значение. При этом дедукция типа ведет себя таким же образом, как дедукция обычного ссылочного параметра функции на l-значение. Выведенный тип для параметра `T` — это тип r-значения:

```
template <typename T> void f3(T&&);
f3(42); // аргумент - это r-значение типа int; параметр
// шаблона T - это int
```

Сворачивание ссылок и параметры ссылок на r-значения

Предположим, что `i` является объектом типа `int`. Можно подумать, что такой вызов, как `f3(i)`, будет недопустим. В конце концов, `i` — это l-значение, а ссылку на r-значение обычно нельзя связать с l-значением. Однако язык определяет два исключения из обычных правил привязки, которые позволяют это. На этих исключениях из правил основан принцип работы таких библиотечных функций, как `move()`.

Первое исключение относится к дедукции типа для ссылочного параметра на r-значение. Когда l-значение (например, `i`) передается параметру функции, являющемуся ссылкой на r-значение на параметр типа шаблона (например, `T&&`), компилятор выводит параметр типа шаблона как тип ссылки на l-значение аргумента. Поэтому, когда происходит вызов `f3(i)`, компилятор выводит тип `T` как `int&`, а не `int`.

Выведение типа `T` как `int&`, казалось бы, означает, что параметр функции `f3()` будет ссылкой на r-значение типа `int&`. Обычно нельзя (непосредственно) определить ссылку на ссылку (см. раздел 2.3.1, стр. 86). Но это можно сделать косвенно, через псевдоним типа (см. раздел 2.5.1, стр. 106) или через параметр типа шаблона.

C++
11

В таких ситуациях проявляется второе исключение из обычных правил привязки: если косвенно создать ссылку на ссылку, то эти ссылки “сворачиваются” (collapse). Во всех случаях кроме одного сворачивание ссылок формирует обычный тип ссылки на l-значение. Новый стандарт дополняет правила свертывания, включая ссылки на r-значение. Ссылки сворачиваются, формируя ссылку на r-значение только в специфическом случае ссылки на r-значение на ссылку на r-значение. Таким образом, для данного типа X:

- X& &, X& && и X&& & сворачиваются в тип X&.
- Тип X&& && сворачивается в тип X&&.



Сворачивание ссылок применимо только тогда, когда ссылка на ссылку создается косвенно, как в псевдониме типа или параметре шаблона.

Комбинация правил свертывания ссылок и специального правила дедукции типа для ссылочных на r-значения параметров означает, что можно вызывать функцию `f3()` для l-значения. Когда параметру функции `f3()` (ссылке на r-значение) передается l-значение, компилятор выведет тип T как тип ссылки на l-значение:

```
f3(i); // аргумент - это l-значение; параметр T шаблона - это int&
f3(ci); // аргумент - это l-значение; параметр T шаблона - это const int&
```

Когда параметр T шаблона выводится как ссылочный тип, правило свертывания гласит, что параметр функции T&& сворачивается в тип ссылки на l-значение. Например, результирующий экземпляр для вызова `f3(i)` получится примерно таким:

```
// недопустимый код, приведен только для примера
void f3<int&>(int& &&); // когда T - это int&, параметр
                        // функции - это int& &&
```

Параметр функции `f3()` — это T&&, а T — это int&, таким образом, T&& будет int& &&, что сворачивается в int&. Таким образом, даже при том, что формой параметра функции `f3()` будет ссылка на r-значение (т.е. T&&), этот вызов создаст экземпляр функции `f3()` с типом ссылки на l-значение (т.е. int&):

```
void f3<int&>(int&); // когда T - это int&, параметр функции
                    // сворачивается в int&
```

У этих правил есть два важных следствия.

- Параметр функции, являющийся ссылкой на r-значение для параметра типа шаблона (например, T&&), может быть связан с l-значением.

- Если аргумент будет l-значением, то выведенный тип аргумента шаблона будет типом ссылки на l-значение, и экземпляр параметра функции будет создан как (обычный) параметр ссылки на l-значение (T&).

Стоит также обратить внимание на то, что параметру функции T&& косвенно можно передать аргумент любого типа. Параметр такого типа может использоваться с r-значениями, а, как было продемонстрировано только что, также и с l-значениями.



Параметру функции, являющемуся ссылкой на r-значение на тип параметра шаблона (т.е. T&&), может быть передан аргумент любого типа. Когда такому параметру передается l-значение, экземпляр параметра функции создается как обычная ссылка на l-значение (T&).

Шаблоны функций с параметрами ссылки на r-значения

У того факта, что параметр шаблона может быть выведен как ссылочный тип, имеются удивительные последствия для кода в шаблоне:

```
template <typename T> void f3(T&& val)
{
    T t = val; // копировать или привязать ссылку?
    t = fcn(t); // изменит ли присвоение только t или val и t?
    if (val == t) { /* ... */ } // всегда истинно, если T - ссылочный тип
}
```

Когда вызов функции f3() происходит для такого r-значения, как литерал 42, T имеет тип int. В данном случае локальная переменная t имеет тип int и инициализируется при копировании значения параметра val. При присвоении переменной t параметр val остается неизменным.

С другой стороны, когда происходит вызов функции f3() для l-значения i, типом T будет int&. Когда определяется и инициализируется локальная переменная t, у нее будет тип int&. Инициализация переменной t свяжет ее с параметром val. При присвоении переменной t одновременно изменяется и параметр val. В этом экземпляре функции f3() оператор if всегда будет возвращать значение true.

На удивление сложно написать правильный код, когда задействованные типы могут быть простыми (не ссылочными) типами или ссылочными типами (хотя такие классы трансформации типов, как remove_reference (см. раздел 16.2.3, стр. 860), вполне могут помочь в этом).

На практике параметры в виде ссылки на r-значение используются в одном из двух случаев: либо когда шаблон перенаправляет свои аргументы, ли когда шаблон перегружается. Перенаправление рассматривается в разделе 16.2.7 (стр. 870), а перегрузка шаблона в разделе 16.3 (стр. 873), а пока достаточно знать, что стоит обратить внимание на то, что шаблоны функций, исполь-

зующие ссылки на *r*-значение, зачастую используют перегрузку таким же образом, как описано в разделе 13.6.3 (стр. 690):

```
template <typename T> void f(T&&);           // привязка к не константным
                                           // r-значениям
template <typename T> void f(const T&);      // l-значения и константные
                                           // r-значения
```

Подобно нешаблонным функциям, первая версия будет связана с изменяемым *r*-значением, а вторая с *l*-значением или константным *r*-значением.

Упражнения раздела 16.2.5

Упражнение 16.42. Определите типы *T* и *val* в каждом из следующих вызовов:

```
template <typename T> void g(T&& val);
int i = 0; const int ci = i;
(a) g(i);      (b) g(ci);      (c) g(i * ci);
```

Упражнение 16.43. Используя определенную в предыдущем упражнении функцию, укажите, каким будет параметр шаблона *g()* при вызове *g(i = ci)*?

Упражнение 16.44. Используя те же три вызова, что и в первом упражнении, определите типы *T*, если параметр функции *g()* объявляется как *T* (а не *T&&*) и как *const T&*?

Упражнение 16.45. С учетом следующего шаблона объясните происходящее при вызове функции *g()* с таким литеральным значением, как 42, и с переменной типа *int*?

```
template <typename T> void g(T&& val) { vector<T> v; }
```



16.2.6. Функция `std::move()`

Библиотечная функция *move()* (см. раздел 13.6.1, стр. 675) — хороший пример шаблона, использующего ссылки на *r*-значение. К счастью, функцию *move()* можно использовать, не понимая механизма работы используемого ею шаблона. Однако изучение работы функции *move()* может помочь понять и использовать шаблоны.

В разделе 13.6.2 (стр. 676) обращалось внимание на то, что, хотя и нельзя непосредственно привязать ссылку на *r*-значение к *l*-значению, функцию *move()* можно использовать для получения ссылки на *r*-значение, связанной с *l*-значением. Поскольку функция *move()* может получать аргументы, по существу, любого типа, нет ничего удивительного в том, что *move()* — это шаблон функции.

Как определена функция `std::move()`

Стандартное определение функции `move()` таково:

```
// об использовании typename в типе возвращаемого значения и
// приведении см. раздел 16.1.3 (стр. 844)
// remove_reference рассматривается в разделе 16.2.3 (стр. 860)
template <typename T>
typename remove_reference<T>::type&& move(T&& t)
{
    // static_cast рассматривается в разделе 4.11.3 (стр. 224)
    return static_cast<typename remove_reference<T>::type&&>(t);
}
```

Этот код короток, но сложен. В первую очередь, параметр функции `move()`, `T&&` является ссылкой на `r`-значение типа параметра шаблона. Благодаря сворачиванию ссылок этот параметр может соответствовать аргументу любого типа. В частности, функции `move()` можно передать либо `l`-, либо `r`-значение:

```
string s1("hi!"), s2;
s2 = std::move(string("bye!")); // ok: перемещение r-значения
s2 = std::move(s1); // ok: но после присвоения значение s1 неопределенно
```

Как работает функция `std::move()`

В первом присвоении аргумент функции `move()` является `r`-значением, полученным в результате выполнения конструктора `string("bye")` класса `string`. Как уже упоминалось, при передаче `r`-значения ссылочному `r`-значению параметра функции выведенный из этого аргумента тип является ссылочным типом (см. раздел 16.2.5, стр. 863). Таким образом, в вызове `std::move(string("bye!"))`:

- выведенным типом `T` будет `string`;
- следовательно, экземпляр шаблона `remove_reference` создается с типом `string`;
- тип-член `type` класса `remove_reference<string>` будет иметь тип `string`;
- типом возвращаемого значения функции `move()` будет `string&&`;
- у параметра `t` функции `move()` будет тип `string&&`

Соответственно, этот вызов создает экземпляр `move<string>`, являющийся следующей функцией:

```
string&& move(string &&t)
```

Тело этой функции возвращает тип `static_cast<string&&>(t)`. Типом `t` уже является `string&&`, поэтому приведение не делает ничего. Следовательно, результатом этого вызова будет ссылка на `r`-значение, которое было дано.

Теперь рассмотрим второе присвоение, которое вызывает функцию `std::move(s1)`. В этом вызове аргументом функции `move()` является l-значение. Поэтому на сей раз:

- выведенным типом `T` будет `string&` (ссылка на тип `string`, а не просто `string`);
- следовательно, экземпляр шаблона `remove_reference` создается с типом `string&`;
- тип-член `type` класса `remove_reference<string&>` будет иметь тип `string`;
- типом возвращаемого значения функции `move()` все еще будет `string&&`;
- параметр `t` функции `move()` будет создан как экземпляр `string&` &&, который сворачивается в `string&`.

Таким образом, этот вызов создает экземпляр шаблона `move<string&>`, который является точно тем, что необходимо для связи ссылки на r-значение с l-значением.

```
string&& move(string &t)
```

Тело этого экземпляра возвращает тип `static_cast<string&&>(t)`. В данном случае типом `t` является `string&`, который приведение преобразует в тип `string&&`.

Оператор `static_cast` поддерживает приведение l-значения к ссылке на r-значение

**C++
11**

Обычно оператор `static_cast` может выполнить только доступные преобразования (см. раздел 16.3, стр. 223). Однако для ссылок на r-значение есть специальное разрешение: даже при том, что нельзя неявно преобразовать l-значение в ссылку на r-значение, используя оператор `static_cast`, можно *явно* привести l-значение к ссылке на r-значение.

Привязка ссылки на r-значение к l-значению создает код, который работает с разрешением ссылке на r-значение заменять l-значение. Иногда, как в случае с функцией `reallocate()` класса `StrVec` (см. раздел 13.6.1, стр. 675), известно, что замена l-значения безопасна. Разрешая осуществлять это приведение, язык позволяет его использование. Вынуждая использовать приведение, язык пытается предотвратить его случайное использование.

И наконец, хотя такие приведения можно написать непосредственно, намного проще использовать библиотечную функцию `move()`. Кроме того, использование функции `std::move()` существенно облегчает поиск в коде места, потенциально способного заменить l-значения.

Упражнения раздела 16.2.6

Упражнение 16.46. Объясните, что этот цикл из функции `StrVec::reallocate()` (раздел 13.5, стр. 671):

```
for (size_t i = 0; i != size(); ++i)
    alloc.construct(dest++, std::move(*elem++));
```



16.2.7. Перенаправление

Некоторые функции должны перенаправлять другим функциям один или несколько своих аргументов с *неизменными* типами. В таких случаях необходимо сохранить всю информацию о перенаправленных аргументах, включая то, является ли тип аргумента константой и является ли аргумент l- или r-значением.

В качестве примера напомним функцию, получающую вызываемое выражение и два дополнительных аргумента. Функция вызовет предоставленное вызываемое выражение с другими двумя аргументами в обратном порядке. Вот первый фрагмент функции обращения:

```
// шаблон, получающий вызываемое выражение и два параметра
// вызывает предоставленное выражение с "обращенными" параметрами
// flip1 - неполная реализация: спецификатор const верхнего уровня и
// ссылки теряются
template <typename F, typename T1, typename T2>
void flip1(F f, T1 t1, T2 t2)
{
    f(t2, t1);
}
```

Этот шаблон работает прекрасно, пока он не используется для вызова функции со ссылочным параметром:

```
void f(int v1, int &v2) // обратите внимание, v2 - ссылка
{
    cout << v1 << " " << ++v2 << endl;
}
```

Здесь функция `f()` изменяет значение аргумента, привязанного к параметру `v2`. Но если происходит вызов функции `f()` через шаблон `flip1`, внесенные функцией `f()` изменения не затронут первоначальный аргумент:

```
f(42, i); // f () изменяет свой аргумент i
flip1(f, j, 42); // вызов f() через flip1 оставляет j неизменным
```

Проблема в том, что `j` передается параметру `t1` шаблона `flip1`. Этот параметр имеет простой, не ссылочный тип `int`, а не `int&`. Таким образом, этот вызов создает следующий экземпляр шаблона `flip1`:

```
void flip1(void(*fcn)(int, int&), int t1, int t2);
```

Значение `j` копируется в `t1`. Ссылочный параметр в функции `f()` связан с `t1`, а не с `j`.



Определение параметров функции, хранящих информацию типа

Чтобы передать ссылку через функцию, необходимо переписать ее так, чтобы параметры сохраняли принадлежность своих аргументов к l-значениям. Немного поразмыслив, можно предположить, что константность аргументов также необходимо сохранить.

Всю информацию о типе аргумента можно сохранить, определив соответствующий ему параметр функции как ссылку на r-значение параметра типа шаблона. Использование ссылочного параметра (l- или r-значение) позволяет сохранить константность, поскольку спецификатор `const` в ссылочном типе нижнего уровня. Благодаря сворачиванию ссылок (см. раздел 16.2.5, стр. 864), если определить параметры функции как `T1&&` и `T2&&`, можно сохранить принадлежность к l- или r-значениям аргументов функции (см. раздел 16.2.5, стр. 863):

```
template <typename F, typename T1, typename T2>
void flip2(F f, T1 &&t1, T2 &&t2)
{
    f(t2, t1);
}
```

Как и прежде, если происходит вызов `flip2(f, j, 42)`, l-значение `j` передается параметру `t1`. Однако в функции `flip2()` для `T1` выводится тип `int&`, а значит, тип `t1` сворачивается в `int&`. Ссылка `t1` связана с `j`. Когда функция `flip2()` вызывает функцию `f()`, ссылочный параметр `v2` в функции `f()` привязан к `t1`, который, в свою очередь, привязан к `j`. Когда функция `f()` осуществляет инкремент `v2`, это изменяет значение `j`.



Параметр функции, являющийся ссылкой на r-значение параметра типа шаблона (т.е. `T&&`), сохраняет константность и принадлежность к l- или r-значениям соответствующих ему аргументов.

Эта версия функции `flip2()` решает одну половину проблемы. Она работает прекрасно с функциями, получающими ссылки на l-значение, но неприменима для вызова функций с параметрами ссылок на r-значение. Например:

```
void g(int &i, int& j)
{
    cout << i << " " << j << endl;
}
```

Если попытаться вызывать функцию `g()` через функцию `flip2()`, то для параметра ссылки на r-значение функции `g()` будет передан параметр `t2`. Даже если функции `flip2()` было передано r-значение, функции `g()` будет передан параметр, носящий в функции `flip2()` имя `t2`:

```
flip2(g, i, 42); // ошибка: нельзя инициализировать int&& из l-значения
```

Параметр функции, как и любая другая переменная, является выражением l-значения (см. раздел 13.6.1, стр. 675). В результате вызов функции `g()` в функции `flip2()` передает l-значение параметру ссылки на r-значение функции `g()`.



Использование функции `std::forward()` для сохранения информации типа в вызове



Чтобы передать функции `flip2()` параметры способом, сохраняющим типы первоначальных аргументов, можно использовать новую библиотечную функцию `forward()`. Как и функция `move()`, функция `forward()` определяется в заголовке `utility`. В отличие от функции `move()`, функцию `forward()` следует вызывать с явным аргументом шаблона (см. раздел 16.2.2, стр. 857). Для этого явного аргумента типа функция `forward()` возвращает ссылку на r-значение. Таким образом, типом возвращаемого значения функции `forward<T>` будет `T&&`.

Обычно функцию `forward()` используют для передачи параметра функции, который определен как ссылка на r-значение, параметру типа шаблона. Благодаря сворачиванию ссылок для типа возвращаемого значения функция `forward()` сохраняет характер (l- или r-значение) переданного ей аргумента:

```
template <typename Type> intermediary(Type &&arg)
{
    finalFcn(std::forward<Type>(arg));
    // ...
}
```

Здесь `Type` используется как тип явного аргумента шаблона функции `forward()` (выводимый из `arg`). Поскольку `arg` — это ссылка на r-значение для параметра типа шаблона, параметр `Type` представит всю информацию типа в аргументе, переданном параметру `arg`. Если этот аргумент будет r-значением, то параметр `Type` будет иметь обычный (не ссылочный) тип и функция `forward<Type>()` возвратит `Type&&`. Если аргумент будет l-значением, то (благодаря сворачиванию ссылок) типом параметра `Type` будет ссылка на l-значение. В данном случае типом возвращаемого значения будет ссылка на r-значение для типа ссылки на l-значение. Снова благодаря сворачиванию ссылок (на сей раз для типа возвращаемого значения) функция `forward<Type>()` возвратит тип ссылки на l-значение.



При использовании с параметром функции, являющимся ссылкой на r-значение для параметра типа шаблона (`T&&`), функция `forward()` сохраняет все подробности типа аргумента.

Перепишем первоначальную функцию, используя на этот раз функцию `forward()`:

```
template <typename F, typename T1, typename T2>
void flip(F f, T1 &&t1, T2 &&t2)
{
    f(std::forward<T2>(t2), std::forward<T1>(t1));
}
```

Если происходит вызов функции `flip(g, i, 42)`, то параметр `i` будет передан функции `g()`, поскольку `int&` и `42` будут переданы как `int&&`.



Подобно функции `std::move()`, для функции `std::forward()` не стоит предоставлять объявление `using`. Причина описана в разделе 18.2.3 (стр. 997).

Упражнения раздела 16.2.7

Упражнение 16.47. Напишите собственную версию функции обращения и проверьте ее, вызывая функции с параметрами ссылок на `r`-значение и `l`-значение.



16.3. Перегрузка и шаблоны

Шаблоны функций могут быть перегружены другими шаблонами или обычными, не шаблонными функциями. Как обычно, функция с тем же именем должна отличаться либо количеством, либо типом своих параметров.

На подбор функции (см. раздел 6.4, стр. 305) присутствие шаблона функции влияет следующими способами.

- В набор функций-кандидатов на вызов включаются любые экземпляры шаблона функции, для которой успешна дедукция аргумента шаблона (см. раздел 16.2, стр. 853).
- Шаблоны функций-кандидатов всегда подходящие, поскольку дедукция аргумента шаблона устранил все неподходящие шаблоны.
- Как обычно, подходящие функции (шаблонные и не шаблонные) ранжируются по преобразованиям, если таковые вообще имеются. Конечно, набор применимых преобразований при вызове шаблона функции весьма ограничен (см. раздел 16.2.1, стр. 854).
- Так же как обычно, если только одна функция обеспечивает наилучшее соответствие, она и выбирается. Но если одинаково хорошее соответствие обеспечивают несколько функций, то:
 - если в наборе одинаково хороших соответствий есть только одна не шаблонная функция, то выбрана будет она;

- если в наборе нет не шаблонных функций, но есть несколько шаблонных, и одна из них более специализированна, чем любые другие, то будет выбран более специализированный шаблон функции;
- в противном случае вызов неоднозначен.



Правильное определение набора перегруженных шаблонов функций требует хорошего понимания отношений между типами и ограничений на преобразования, применимых к аргументам в шаблонах функций.

Создание перегруженных шаблонов

В качестве примера создадим набор функций, которые могли бы пригодиться во время отладки. Назовем отладочные функции `debug_rep()`, каждая из них возвратит строковое представление предоставленного объекта. Начнем с создания самой общей версии этой функции в качестве шаблона, получающего ссылку на константный объект:

```
// выводит любой тип, который иначе не обработать
template <typename T> string debug_rep(const T &t)
{
    ostringstream ret; // см. раздел 8.3 (стр. 413)
    ret << t; // использует оператор вывода T для вывода представления t
    return ret.str(); // возвращает копию строки, с которой связан ret
}
```

Эта функция применяется для создания строки, соответствующей объекту любого типа, у которого есть оператор вывода.

Теперь определим версию функции `debug_rep()` для вывода указателя:

```
// выводит указатели как их значение, сопровождаемое объектом, на который
// он указывает
// обратите внимание: эта функция не будет работать правильно с char*;
// см. раздел 16.3 (стр. 877)
template <typename T> string debug_rep(T *p)
{
    ostringstream ret;
    ret << "pointer: " << p; // выводит собственное значение указателя
    if (p)
        ret << " " << debug_rep(*p); // выводит значение, на которое
        // указывает p
    else
        ret << " null pointer"; // или указывают, что p - нулевой
    return ret.str(); // возвращает копию строки, с которой связан ret
}
```

Эта версия создает строку, содержащую собственное значение указателя и вызывает функцию `debug_rep()` для вывода объекта, на который указывает этот указатель. Обратите внимание, что эта функция не может использоваться для вывода символьных указателей, поскольку библиотека ввода-вывода опре-

деляет версию оператора `<<` для значения указателя `char*`. Эта версия оператора `<<` подразумевала, что указатель обозначает символьный массив с нулевым символом в конце и выводит содержимое массива, а не его адрес. Обработка символьных указателей рассматривается в разделе 16.3 (стр. 877).

Эти функции можно использовать следующим образом:

```
string s("hi");
cout << debug_rep(s) << endl;
```

Подходящей для этого вызова является только первая версия функции `debug_rep()`. Второй версии требуется параметр в виде указателя, а в этом вызове передан не указатель. Нет никакого способа создать экземпляр шаблона функции, ожидающего тип указателя, из параметра, который не является указателем, поэтому дедукция аргумента терпит неудачу. Поскольку есть только одна подходящая функция, она и используется.

Если происходит вызов функции `debug_rep()` с указателем:

```
cout << debug_rep(&s) << endl;
```

то обе функции создают подходящие экземпляры:

- `debug_rep(const string*&)` — экземпляр первой версии функции `debug_rep()` с привязкой параметра `T` к типу `string*`;
- `debug_rep(string*)` — экземпляр второй версии функции `debug_rep()` с привязкой параметра `T` к типу `string`.

Точным соответствием для этого вызова является экземпляр второй версии функции `debug_rep()`. Создание экземпляра первой версии требует преобразования простого указателя в указатель на константу. Обычный подбор функции гласит, что следует предпочесть второй шаблон, и в действительности так и происходит.

Несколько подходящих шаблонов

В качестве другого примера рассмотрим следующий вызов:

```
const string *sp = &s;
cout << debug_rep(sp) << endl;
```

Здесь подходящими являются оба шаблона, и оба обеспечивают точное соответствие:

- `debug_rep(const string*&)` — экземпляр первой версии шаблона с привязкой параметра `T` к типу `const string*`;
- `debug_rep(const string*)` — экземпляр второй версии шаблона с привязкой параметра `T` к типу `const string`.

В данном случае обычный подбор функции не может различить эти два вызова. Можно было бы ожидать, что этот вызов будет неоднозначен. Однако

благодаря специальному правилу для перегруженных шаблонов функций этот вызов решается как `debug_rep(T*)`, поскольку это более специализированный шаблон.

Причина для этого правила в том, что без него не было бы никакого способа вызвать версию функции `debug_rep()` для указателя на константу. Проблема в том, что к шаблону `debug_rep(const T&)` подходит практически любой тип, включая типы указателя. Этот шаблон является более общим, чем `debug_rep(T*)`, который может быть вызван только для типов указателя. Без этого правила вызовы с передачей указателей на константу всегда будут неоднозначны.



Когда есть несколько перегруженных шаблонов, предоставляющих одинаково хорошее соответствие для вызова, предпочитается наиболее специализированная версия.

Не шаблон и перегрузка шаблона

Для следующего примера определим обычную, не шаблонную версию функции `debug_rep()`, выводящую строки в двойных кавычках:

```
// вывод строк в двойных кавычках
string debug_rep(const string &s)
{
    return '"' + s + '"';
}
```

Теперь, когда происходит вызов функции `debug_rep()` для строки:

```
string s("hi");
cout << debug_rep(s) << endl;
```

есть две одинаково хорошо подходящих функции:

- `debug_rep<string>(const string&)` — первый шаблон с привязкой параметра `T` к типу `string`;
- `debug_rep(const string&)` — обычная, не шаблонная функция.

В данном случае у обеих функций одинаковый список параметров, поэтому каждая из них обеспечивает одинаково хорошее соответствие этому вызову. Однако выбирается не шаблонная версия. По тем же причинам, по которым предпочитают наиболее специализированные из одинаково хорошо подходящих шаблонов функций, не шаблонная функция предпочитается при одинаково хорошем соответствии с шаблонной функцией.



Когда не шаблонная функция обеспечивает одинаково хорошее соответствие с шаблонной функцией, предпочитается не шаблонная версия.

Перегруженные шаблоны и преобразования

До сих пор не рассматривался случай с указателями на символьные строки в стиле C и строковые литералы. Теперь, когда имеется версия функции `debug_rep()`, получающая строку, можно было бы ожидать, что ей будет соответствовать вызов, которому переданы символьные строки. Однако рассмотрим этот вызов:

```
cout << debug_rep("hi world!") << endl; // вызов debug_rep(T*)
```

Здесь подходящими являются все три функции `debug_rep()`:

- `debug_rep(const T&)` — с привязкой параметра `T` к типу `char[10]`;
- `debug_rep(T*)` — с привязкой параметра `T` к типу `const char`;
- `debug_rep(const string&)` — требующая преобразования из `const char*` в `string`.

Оба шаблона обеспечивают точное соответствие аргументу — второй шаблон требует (допустимого) преобразования из массива в указатель, и это преобразование считается точным соответствием при подборе функции (см. раздел 6.6.1, стр. 320). Не шаблонная версия является подходящей, но требует пользовательского преобразования. Эта функция хуже точного соответствия, поэтому кандидатами остаются два шаблона. Как и прежде, версия `T*` более специализирована, она и будет выбрана.

Если символьные указатели необходимо обработать как строки, можно определить еще две перегруженные, не шаблонные функции:

```
// преобразовать символьные указатели в строку и вызвать строковую
// версию debug_rep()
string debug_rep(char *p)
{
    return debug_rep(string(p));
}
string debug_rep(const char *p)
{
    return debug_rep(string(p));
}
```

Пропуск объявления может нарушить программу

Следует заметить, что для правильной работы версии `char*` функции `debug_rep()` объявление `debug_rep(const string&)` должно находиться в области видимости, когда эти функции определяются. В противном случае будет вызвана неправильная версия функции `debug_rep()`:

```
template <typename T> string debug_rep(const T &t);
template <typename T> string debug_rep(T *p);
// следующее объявление должно быть в области видимости
// для правильного определения debug_rep(char *)
```

```
string debug_rep(const string &);
string debug_rep(char *p)
{
    // если объявление для версии, получающей const string&, не находится
    // в области видимости, return вызовет call debug_rep(const T&) с
    // экземпляром строки в параметре T
    return debug_rep(string(p));
}
```

Обычно, если попытаться использовать функцию, которую забыли объявлять, код не будет откомпилирован. Но с функциями, которые перегружают шаблон функции, все не так. Если компилятор может создать экземпляр вызова из шаблона, то отсутствие объявления не будет иметь значения. В этом примере, если забыть объявлять версию функции `debug_rep()`, получающую строку, компилятор *тихо* создаст версию экземпляра шаблона, получающую `const T&`.



Объявляйте каждую функцию в наборе перегруженных, прежде чем определять их. Таким образом можно гарантировать, что компилятор создаст экземпляр вызова прежде, чем он встретит функцию, которую предполагалось вызвать.

Упражнения раздела 16.3

Упражнение 16.48. Напишите собственные версии функций `debug_rep()`.

Упражнение 16.49. Объясните, что происходит в каждом из следующих вызовов:

```
template <typename T> void f(T);
template <typename T> void f(const T*);
template <typename T> void g(T);
template <typename T> void g(T*);
int i = 42, *p = &i;
const int ci = 0, *p2 = &ci;
g(42);    g(p);    g(ci);    g(p2);
f(42);    f(p);    f(ci);    f(p2);
```

Упражнение 16.50. Определите функции из предыдущего упражнения так, чтобы они выводили идентификационное сообщение. Выполните код этого упражнения. Если вызовы ведут себя не так, как ожидалось, выясните почему.



16.4. Шаблоны с переменным количеством аргументов



Шаблон с переменным количеством аргументов (variadic template) — это шаблон функции или класса, способный получать переменное количество параметров. Набор таких параметров называется *пакетом параметров* (parameter pack). Есть два вида пакетов параметров: *пакет параметров*

шаблона (template parameter pack), представляющий любое количество параметров шаблона, и *пакет параметров функции* (function parameter pack), представляющий любое количество параметров функции.

Для указания, что шаблону или функции представлен пакет параметров, используется многоточие. В списке параметров шаблона синтаксис `class...` или `typename...` означает, что следующий параметр представляет список любого количества типов; имя типа, сопровождаемое многоточием, представляет список из любого количества параметров значения заданного типа. Параметр в списке параметров функции, типом которого является пакет параметров шаблона, представляет собой пакет параметров функции. Например:

```
// Args - это пакет параметров шаблона; rest - пакет параметров функции
// Args представляет любое количество параметров типа шаблона
// rest представляет любое количество параметров функции
template <typename T, typename... Args>
void foo(const T &t, const Args& ... rest);
```

Этот код объявляет, что `foo()` — это функция с переменным количеством аргументов, у которой один параметр типа `T` по имени `T` и пакет параметров шаблона по имени `Args`. Этот пакет представляет любое количество дополнительных параметров типа. В списке параметров функции `foo()` один параметр типа `const T&` для любого типа переданного параметром `T` и пакет параметров функции `rest`. Этот пакет представляет любое количество параметров функции.

Как обычно, компилятор выводит типы параметра шаблона из аргументов функции. Для шаблона с переменным количеством аргументов компилятор также выводит количество параметров в пакете. Рассмотрим, например, следующие вызовы:

```
int i = 0; double d = 3.14; string s = "how now brown cow";
foo(i, s, 42, d); // три параметра в пакете
foo(s, 42, "hi"); // два параметра в пакете
foo(d, s);        // один параметр в пакете
foo("hi");        // пустой пакет
```

Компилятор создаст четыре разных экземпляра функции `foo()`:

```
void foo(const int&, const string&, const int&, const double&);
void foo(const string&, const int&, const char[3]&);
void foo(const double&, const string&);
void foo(const char[3]&);
```

В каждом случае тип `T` выводится из типа первого аргумента. Остальные аргументы (если они есть) представляют количество и типы дополнительных аргументов функции.

Оператор `sizeof...`



Когда необходимо узнать, сколько элементов находится в пакете, можно использовать оператор `sizeof...`. Как и оператор `sizeof` (см. раздел 4.9, стр. 215), оператор `sizeof...` возвращает константное выражение (см. раздел 2.4.4, стр. 103) и не вычисляет свой аргумент:

```
template<typename ... Args> void g(Args ... args) {
    cout << sizeof...(Args) << endl; // количество параметров типа
    cout << sizeof...(args) << endl; // количество параметров функции
}
```

Упражнения раздела 16.4

Упражнение 16.51. Определите, что возвратят операторы `sizeof...(Args)` и `sizeof...(rest)` для каждого вызова функции `foo()` в этом разделе.

Упражнение 16.52. Напишите программу, проверяющую ответы на предыдущий вопрос.



16.4.1. Шаблоны функции с переменным количеством аргументов

В разделе 6.2.6 (стр. 290) упоминалось, что для определения функции, способной получать переменное количество аргументов, можно использовать класс `initializer_list`. Однако у аргументов должен быть одинаковый тип (или типы, преобразуемые в общий тип). Функции с переменным количеством аргументов используются тогда, когда не известно ни количество, ни типы аргументов. Для примера определим функцию, подобную прежней функции `error_msg()`, только на сей раз обеспечим и изменение типов аргумента. Начнем с определения функции `print()` с переменным количеством аргументов, которая выводит содержимое заданного списка аргументов в указанный поток.

Функции с переменным количеством аргументов зачастую рекурсивны (см. раздел 6.3.2, стр. 298). Первый вызов обрабатывает первый аргумент в пакете и вызывает себя для остальных аргументов. Новая функция `print()` будет работать таким же образом — каждый вызов выводит свой второй аргумент в поток, обозначенный первым аргументом. Для остановки рекурсии следует определить также обычную функцию `print()`, которая получает поток и объект:

```
// Функция для завершения рекурсии и вывода последнего элемента
// ее следует объявить перед определением версией print() с переменным
// количеством аргументов
template<typename T>
ostream &print(ostream &os, const T &t)
```

```
{
    return os << t; // нет разделителя после последнего элемента в пакете
}
// эта версия print() будет вызвана для всех элементов в пакете, кроме
// последнего
template <typename T, typename... Args>
ostream &print(ostream &os, const T &t, const Args&... rest)
{
    os << t << ", "; // выводит первый аргумент
    return print(os, rest...); // рекурсивный вызов; вывод других
                               // аргументов
}
```

Первая версия функции `print()` останавливает рекурсию и выводит последний аргумент в начальном вызове функции `print()`. Вторая версия, с переменным количеством аргументов, выводит аргумент, связанный с `t`, и вызывает себя для вывода остальных значений в пакете параметров функции.

Ключевая часть — вызов функции `print()` в функции с переменным количеством аргументов:

```
return print(os, rest...); // рекурсивный вызов; вывод других
                           // аргументов
```

Версия функции `print()` с переменным количеством аргументов получает три параметра: `ostream&`, `const T&` и пакет параметров. Но в этом вызове передаются только два аргумента. В результате первый аргумент в пакете `rest` привязывается к `t`. Остальные аргументы в пакете `rest` формируют пакет параметров для следующего вызова функции `print()`. Таким образом, при каждом вызове первый аргумент удаляется из пакета и становится аргументом, связанным с `t`. Соответственно, получаем:

```
print(cout, i, s, 42); // два параметра в пакете
```

Рекурсия выполнится следующим образом:

Вызов	t	rest...
print(cout, i, s, 42)	i	s, 42
print(cout, s, 42)	s	42

Вызов `print(cout, 42)` вызывает обычную версию функции `print()`.

Первые два вызова могут соответствовать только версии функции `print()` с переменным количеством аргументов, поскольку обычная версия не является подходящей. Эти вызовы передают четыре и три аргумента соответственно, а обычная функция `print()` получает только два аргумента.

Для последнего вызова в рекурсии, `print(cout, 42)`, подходят обе версии функции `print()`. Этот вызов передает два аргумента, и типом первого являются `ostream&`. Таким образом, подходящей является обычная версия функции `print()`.

Версия с переменным количеством аргументов также является подходящей. В отличие от обычного аргумента, пакет параметров может быть пустым. Следовательно, экземпляр версии функции `print()` с переменным количеством аргументов может быть создан только с двумя параметрами: один — для параметра `ostream&` и другой — для параметра `const T&`.

Обе функции обеспечивают одинаково хорошее соответствие для вызова. Однако не шаблонная версия с переменным количеством аргументов более специализирована, чем шаблонная с переменным количеством аргументов. Поэтому выбирается версия без переменного количества аргументов (см. раздел 16.3, стр. 874).



Объявление версии функции `print()` с постоянным количеством аргументов должно быть в области видимости, когда определяется версия с переменным количеством аргументов. В противном случае функция с переменным количеством аргументов будет рекурсивно вызывать себя бесконечно.

Упражнения раздела 16.4.1

Упражнение 16.53. Напишите собственные версии функций `print()` и проверьте их, выводя один, два и пять аргументов, у каждого из которых должны быть разные типы.

Упражнение 16.54. Что происходит при вызове функции `print()` для типа, не имеющего оператора `<<?`

Упражнение 16.55. Объясните, как выполнялась бы версия функции `print()` с переменным количеством аргументов, если бы обычная версия функции `print()` была объявлена после определения версии с переменным количеством аргументов.



16.4.2. Развертывание пакета

Кроме выяснения размера, единственное, что можно еще сделать с пакетом параметров, — это *развернуть* (pack expansion) его. При развертывании пакета предоставляется *схема* (pattern), используемая для каждого развернутого элемента. Развертывание пакета разделяет его на элементы с применением схемы к каждому из них. Для запуска развертывания справа от схемы помещают многоточие (...).

Например, функция `print()` содержит два развертывания:

```
template <typename T, typename... Args>
ostream &
print(ostream &os, const T &t, const Args&... rest) // развертывание Args
{
```

```

os << t << ", ";
return print(os, rest...); // развертывание rest
}

```

В первом случае развертывание пакета параметров шаблона создает список параметров функции `print()`. Второй случай развертывания находится в вызове функции `print()`. Эта схема создает список аргументов для вызова.

Развертывание пакета `Args` применяет схему `const Args&` к каждому элементу в пакете параметров шаблона `Args`. Результатом этой схемы будет разделенный запятыми список из любого количества типов параметров в формате `const тип&`. Например:

```
print(cout, i, s, 42); // два параметра в пакете
```

Типы последних двух аргументов, наряду со схемой, определяют типы замыкающих параметров. Этот вызов создает следующий экземпляр:

```
ostream&
print(ostream&, const int&, const string&, const int&);
```

Второе развертывание происходит в рекурсивном вызове функции `print()`. В данном случае схема — это имя пакета параметров функции (т.е. `rest`). Эта схема развертывается в разделяемый запятыми список элементов пакета. Таким образом, этот вызов эквивалентен следующему:

```
print(os, s, 42);
```

Концепция развертывания пакета

Развертывание пакета параметров функции `print()` только разворачивало пакет на его составные части. При развертывании пакета параметров функции возможны и более сложные схемы. Например, можно было бы написать вторую функцию с переменным количеством аргументов, которая вызывает функцию `debug_rep()` (см. раздел 16.3, стр. 874) для каждого из своих аргументов, а затем вызывает функцию `print()`, чтобы вывести полученные строки:

```

// вызвать debug_rep() для каждого аргумента в вызове print()
template <typename... Args>
ostream &errorMsg(ostream &os, const Args&... rest)
{
    // print(os, debug_rep(a1), debug_rep(a2), ..., debug_rep(an))
    return print(os, debug_rep(rest)...);
}

```

Вызов функции `print()` использует схему `debug_rep(rest)`. Эта схема означает, что функцию `debug_rep()` следует вызвать для каждого элемента в пакете параметров функции `rest`. Получившийся развернутый пакет будет разделяемым запятыми списком вызовов функции `debug_rep()`. Таким образом, вызов

```
errorMsg(cerr, fcnName, code.num(), otherData, "other", item);
```

выполняется, как будто было написано:

```
print(cerr, debug_rep(fcnName), debug_rep(code.num()),
      debug_rep(otherData), debug_rep("otherData"),
      debug_rep(item));
```

Следующая схема, напротив, не была бы откомпилирована:

```
// передает пакет debug_rep(); print(os, debug_rep(a1, a2, ..., an))
print(os, debug_rep(rest...)); // ошибка: нет функции, соответствующей
                               // вызову
```

Проблема здесь в том, что пакет `rest` развернут в вызове функции `debug_rep()`. Этот вызов выполнен бы так, как будто было написано:

```
print(cerr, debug_rep(fcnName, code.num(),
                     otherData, "otherData", item));
```

В этом развертывании осуществляется попытка вызова функции `debug_rep()` со списком из пяти аргументов. Нет никакой версии функции `debug_rep()`, соответствующей этому вызову. Функция `debug_rep()` имеет постоянное количество аргументов, и нет никакой ее версии с пятью параметрами.



Схема при развертывании применяется по отдельности к каждому элементу в пакете.

Упражнения раздела 16.4.2

Упражнение 16.56. Напишите и проверьте версию функции `errorMsg()` с переменным количеством аргументов.

Упражнение 16.57. Сравните свою версию функции `errorMsg()` с переменным количеством аргументов с функцией `error_msg()` из раздела 6.2.6 (стр. 290). Каковы преимущества и недостатки каждого подхода?



16.4.3. Перенаправление пакетов параметров



По новому стандарту можно использовать шаблоны с переменным количеством аргументов совместно с функцией `forward()` для написания функций, которые передают свои аргументы неизменными некой другой функции. Чтобы проиллюстрировать такие функции, добавим в класс `StrVec` (см. раздел 13.5, стр. 666) функцию-член `emplace_back()`. Такая функция-член библиотечных контейнеров является шаблоном-членом с переменным количеством аргументов (см. раздел 16.1.4, стр. 847), которая использует их для создания элементов непосредственно в области, управляемой контейнером.

Версия функции `emplace_back()` для класса `StrVec` также должна быть с переменным количеством аргументов, поскольку у класса `string` много конструкторов, которые отличаются своими параметрами.

Поскольку желательно быть в состоянии использовать конструктор перемещения класса `string`, необходимо будет также сохранять всю информацию о типах аргументов, переданных функции `emplace_back()`.

Как уже упоминалось, сохранение информации типа — двухступенчатый процесс. Во-первых, для сохранения информации типа аргументов параметры функции `emplace_back()` следует определить как ссылки на `r`-значение параметра типа шаблона (см. раздел 16.2.7, стр. 871):

```
class StrVec {
public:
    template <class... Args> void emplace_back(Args&&...);
    // остальные члены, как в разделе 13.5 (стр. 666)
};
```

Схема `&&` в развертывании пакета параметров шаблона означает, что каждый параметр функции будет ссылкой на `r`-значение на соответствующий ей аргумент.

Во-вторых, функцию `forward()` следует использовать для сохранения первоначальных типов аргументов, когда функция `emplace_back()` передает их функции `construct()` (см. раздел 16.2.7, стр. 872):

```
template <class... Args>
inline
void StrVec::emplace_back(Args&&... args)
{
    chk_n_alloc(); // пересоздает StrVec при необходимости
    alloc.construct(first_free++, std::forward<Args>(args)...);
}
```

Тело функции `emplace_back()` вызывает функцию `chk_n_alloc()` (см. раздел 13.5, стр. 666), чтобы гарантировать наличие достаточного места для элемента, и вызывает функцию `construct()`, чтобы создать элемент в позиции, на которую указывает указатель `first_free`.

```
std::forward<Args>(args)...
```

Развертывание в вызове функции `construct()` разворачивает оба пакета: параметров шаблона `Args` и параметров функции `args`. Эта схема создает элементы в формате:

```
std::forward<Ti>(ti)
```

где T_i представляет тип i -го элемента в пакете параметров шаблона, а t_i представляет i -й элемент в пакете параметров функции. Например, если `svec` имеет тип `StrVec`, то при вызове

```
svec.emplace_back(10, 'c'); // добавит ccccccccc как новый последний
                             // элемент
```

схема в вызове функции `construct()` развернется в

```
std::forward<int>(10), std::forward<char>(c)
```

Использование функции `forward()` в этом вызове гарантирует, что если функция `emplace_back()` будет вызвана с *r*-значением, то функция `construct()` также получит *r*-значение. Например, в вызове

```
svec.emplace_back(s1 + s2); // использует конструктор перемещения
```

аргумент функции `emplace_back()` является *r*-значением, которое передается функции `construct()` как

```
std::forward<string>(string("the end"))
```

Типом результата вызова `forward<string>` будет `string&&`, поэтому функция `construct()` будет вызвана со ссылкой на *r*-значение. Функция `construct()`, в свою очередь, перенаправит этот аргумент конструктору перемещения класса `string`, чтобы создать этот элемент.

СОВЕТ. ПЕРЕНАПРАВЛЕНИЕ И ШАБЛОНЫ С ПЕРЕМЕННЫМ КОЛИЧЕСТВОМ АРГУМЕНТОВ

Функции с переменным количеством аргументов зачастую перенаправляют свои параметры другим функциям. Форма таких функций, как правило, подобна функции `emplace_back()`:

```
// у функции fun() может быть любое количество параметров, каждый
// из которых является ссылкой r-значения на тип параметра шаблона
template<typename... Args>
void fun(Args&&... args) // развертывание Args в список ссылок
                        // на r-значения
{
    // аргумент work() развертывает как Args, так и args
    work(std::forward<Args>(args)...);
}
```

Здесь предполагается перенаправить все аргументы функции `fun()` другой функции, `work()`, которая, по-видимому, осуществляет реальную работу. Как и вызов функции `construct()` в функции `emplace_back()`, развертывание в вызове функции `work()` разворачивает и пакет параметров шаблона, и пакет параметров функции.

Поскольку параметры функции `fun()` являются ссылками на *r*-значение, функции `fun()` можно передать аргументы любого типа; поскольку для передачи этих аргументов используется функция `std::forward()`, вся информация о типах этих аргументов будет сохраняться в вызове функции `work()`.

Упражнения раздела 16.4.3

Упражнение 16.58. Напишите функцию `emplace_back()` для собственного класса `StrVec` и для класса `Vec`, написанного в упражнении раздела 16.1.2 (стр. 841).

Упражнение 16.59. С учетом того, что `s` имеет тип `string`, объясните вызов `svec.emplace_back(s)`.

Упражнение 16.60. Объясните, как работает функция `make_shared()` (см. раздел 12.1.1, стр. 576).

Упражнение 16.61. Определите собственную версию функции `make_shared()`.



16.5. Специализация шаблона

Не всегда можно написать один шаблон, который наилучшим образом подходит для всех возможных типов аргументов шаблона, для которых может быть создан его экземпляр. В некоторых случаях общий шаблон просто не подходит для типа: он либо приводит к ошибке при компиляции, либо к неправильным действиям. С другой стороны, иногда можно воспользоваться уникальными возможностями определенного типа для создания более эффективной функции, чем та, которой снабжен экземпляр общего шаблона.

Функция `compare()` — хороший пример шаблона функции, общее определение которого не подходит для специфического типа, а именно символьных указателей. Хотелось бы, чтобы функция `compare()` сравнивала символьные указатели, используя функцию `strcmp()`, а не сравнивала значения указателей. Действительно, ведь уже есть перегруженная функция `compare()`, обрабатывающая символьные строковые литералы (см. раздел 16.1.1, стр. 824):

```
// первая версия; может сравнить любые два типа
template <typename T> int compare(const T&, const T&);
// вторая версия, для обработки строковых литералов
template<size_t N, size_t M>
int compare(const char (&)[N], const char (&)[M]);
```

Однако версия функции `compare()` с двумя параметрами значения шаблона будет вызвана только при передаче строкового литерала или массива. Если происходит вызов функции `compare()` с символьными указателями, будет вызвана первая версия шаблона:

```
const char *p1 = "hi", *p2 = "mom";
compare(p1, p2);           // вызывает первый шаблон
compare("hi", "mom");      // вызывает шаблон с двумя параметрами значения
```

Нет никакого способа преобразовать указатель в ссылку на массив, поэтому вторая версия функции `compare()` не подходит для передачи указателей `p1` и `p2` как аргументов.

Для обработки символьных указателей (в отличие от массивов) можно опделить *специализацию шаблона* (template specialization) для первой версии функции `compare()`. Специализация — это отдельное определение шаблона, в котором определяется один или несколько параметров шаблона для получения специфического типа.

Специализация шаблона функции

При специализации шаблона функции следует предоставить аргументы для каждого параметра первоначального шаблона. Для указания специализации шаблона используется ключевое слово `template`, сопровождаемое парой пустых угловых скобок (`<>`). Пустые скобки означают, что аргументы будут предоставлены для всех параметров первоначального шаблона:

```
// специальная версия compare() для работы с указателями на символьные
// массивы
template <>
int compare(const char* const &p1, const char* const &p2)
{
    return strcmp(p1, p2);
}
```

Трудная для понимания часть этой специализации относится к типам параметра функции. При определении специализации типы параметров функции должны совпадать с соответствующими типами ранее объявленного шаблона:

```
template <typename T> int compare(const T&, const T&);
```

В этой специализации параметры функции являются ссылками на константные типы. Подобно псевдонимам типа, взаимодействие между типами параметра шаблона, указателями и константами может удивить (см. раздел 2.5.1, стр. 106).

Необходимо определить специализацию шаблона этой функции с типом `const char*` для параметра `T`. Функция потребует ссылки на константную версию этого типа. Константная версия типа указателя — это константный указатель, а не указатель на константу (см. раздел 2.4.2, стр. 100). В данной специализации следует использовать тип `const char* const &`, являющийся ссылкой на константный указатель на константный символ.

Перегрузка функций или специализация шаблона

При определении специализации шаблона функции разработчик, по существу, выполняет задачу компилятора. Таким образом, определение предоставляется для использования специфического экземпляра первоначального шаблона. Важно понимать, что специализация — это создание экземпляра функции; а не перегрузка ее экземпляра.



Специализация создает экземпляр шаблона, а не перегружает его. В результате специализация не затрагивает механизм подбора функций.

Может ли определение некой функции как специализации шаблона или как независимой, не шаблонной функции повлиять на подбор функций? Предположим, например, что имеется определение двух версий шаблонной функции `compare()`: той, что получает параметры как ссылки на массив, и другой, которая получает тип `const T&`. Факт наличия специализации для символьных указателей никак не влияет на подбор функции:

```
compare("hi", "mom")
```

Когда функция `compare()` вызывается для строкового литерала, оба шаблона функции оказываются подходящими и обеспечивают одинаково хорошее (т.е. точное) соответствие вызову. Однако версия с параметрами символьного массива более специализирована (см. раздел 16.3, стр. 874), она и выбирается для этого вызова.

Если бы была определена версия функции `compare()`, получающая указатели на символы, как простая, не шаблонная функция (а не как специализация шаблона), то этот вызов разрешится по-другому. В данном случае было бы три подходящих функции: эти два шаблона и не шаблонная версия указателя на символ. Все три одинаково хорошо подходят для этого вызова. Как уже упоминалось, когда не шаблонная функция обеспечивает одинаково хорошее соответствие с шаблонной, выбирается не шаблонная функция (см. раздел 16.3, стр. 874).

КЛЮЧЕВАЯ КОНЦЕПЦИЯ. ОБЫЧНЫЕ ПРАВИЛА ОБЛАСТИ ВИДИМОСТИ ОТНОСЯТСЯ И К СПЕЦИАЛИЗАЦИИ

Чтобы специализировать шаблон, объявление его оригинала должно быть в области видимости. Кроме того, объявление специализации должно быть в области видимости перед любым кодом, использующим экземпляр шаблона.

Пропуск объявления обычных классов и функций найти очень просто — компилятор не сможет обработать такой код. Но при отсутствии объявления специализации компилятор обычно создает код, используя первоначальный шаблон. Поэтому ошибки в порядке объявления шаблона и его специализации довольно просто допустить, но очень трудно найти.

Использование специализации и экземпляра первоначального шаблона с тем же набором аргументов шаблона является ошибкой. Но компилятор вряд ли обнаружит эту ошибку.

Рекомендуем

Шаблоны и их специализации должны быть объявлены в том же файле заголовка. Объявления всех шаблонов с данным именем должны располагаться сначала, а затем все специализации этих шаблонов.

Специализация шаблона класса

Кроме специализации шаблонов функций, вполне можно также специализировать шаблоны классов. В качестве примера определим специализацию библиотечного шаблона `hash`, который можно использовать для хранения объектов класса `Sales_data` в неупорядоченном контейнере. По умолчанию неупорядоченные контейнеры используют для организации своих элементов класс `hash<key_type>` (см. раздел 11.4, стр. 567). Чтобы использовать его с собственным типом данных, следует определить специализацию шаблона `hash`. Специализированный класс `hash` должен определять следующее.

- Перегруженный оператор вызова (см. раздел 14.8, стр. 722), возвращающий тип `size_t` и получающий объект типа ключа контейнера.
- Два члена-типа `result_type` и `argument_type`, соответствующие типу возвращаемого значения и типу аргумента оператора вызова.
- Стандартный конструктор и оператор присвоения копии, которые могут быть определены неявно (см. раздел 13.1.2, стр. 634).

Единственное осложнение в определении этой специализации класса `hash` состоит в том, что специализация шаблона должна быть в том же пространстве имен, в котором определяется первоначальный шаблон. Более подробная информация о пространствах имен приведена в разделе 18.2 (стр. 981), а пока достаточно знать, что к пространству имен можно добавлять члены. Для этого следует сначала открыть пространство имен:

```
// открыть пространство имен std, чтобы можно было специализировать
// класс std::hash
namespace std {
} // закрыть пространство имен std; обратите внимание: никакой точки с
// запятой после закрывающей фигурной скобки
```

Любые определения, расположенные между открывающей и закрывающей фигурными скобками, будут частью пространства имен `std`.

Следующий код определяет специализацию класса `hash` для класса `Sales_data`:

```
// открыть пространство имен std, чтобы можно было специализировать
// класс std::hash
namespace std {
template <> // определение специализации с параметром
struct hash<Sales_data> // шаблона класса Sales_data
{
    // тип, используемый для неупорядоченного контейнера hash, должен
    // определять следующие типы
    typedef size_t result_type;
    typedef Sales_data argument_type; // по умолчанию этому типу
    // требуется оператор ==
    size_t operator()(const Sales_data& s) const;
    // класс использует синтезируемые функции управления копированием
```

```

    // и стандартный конструктор
};
size_t
hash<Sales_data>::operator()(const Sales_data& s) const
{
    return hash<string>()(s.bookNo) ^
           hash<unsigned>()(s.units_sold) ^
           hash<double>()(s.revenue);
}
// закрыть пространство имен std; обратите внимание: никакой точки с
// запятой после закрывающей фигурной скобки

```

Определение `hash<Sales_data>` начинается с части `template<>`, означающей, что определяется полностью специализированный шаблон. Специализируемый шаблон называется `hash`, а специализированная версия — `hash<Sales_data>`. Члены класса следуют непосредственно из требований для специализации шаблона `hash`.

Подобно любым другим классам, специализируемые члены можно определить в классе или вне его, как это сделано здесь. Перегруженный оператор вызова должен определять хеш-функцию по значениям заданного типа. Эта функция обязана возвращать каждый раз тот же результат, когда она вызывается для данного значения. Хеш-функция практически всегда возвращает другой результат для не равных объектов.

Все сложности определения хорошей хеш-функции делегируем библиотеке. Библиотека определяет специализации класса `hash` для встроенных типов и для большинства библиотечных типов. Безымянный объект `hash<string>` используется для создания хеш-кода для переменной-члена `bookNo`, объект типа `hash<unsigned>` для создания хеш-кода из переменной-члена `units_sold` и объекта типа `hash<double>` для создания хеш-кода из переменной-члена `revenue`. Применение к этим результатам оператора исключающего ИЛИ (см. раздел 4.8, стр. 213) сформирует общий хеш-код для заданного объекта класса `Sales_data`.

Следует заметить, что хеш-функция определена для хеширования всех трех переменных-членов, чтобы она была совместима с определением оператора `operator==` класса `Sales_data` (см. раздел 14.3.1, стр. 711). По умолчанию неупорядоченные контейнеры используют специализацию хеша, соответствующую типу `key_type`, наряду с оператором равенства типа ключа.

С учетом того, что специализация находится в области видимости, она будет использоваться автоматически при использовании класса `Sales_data` как ключ в одном из этих контейнеров:

```

// использует hash<Sales_data> и оператор operator== класса Sales_data
// из раздела 14.3.1 (стр. 711)
unordered_multiset<Sales_data> SDset;

```

Поскольку `hash<Sales_data>` использует закрытые члены класса `Sales_data`, этот класс следует сделать другом класса `Sales_data`:

```
template <class T> class std::hash; // нужно для объявления дружественным
class Sales_data {
friend class std::hash<Sales_data>;
    // другие члены, как прежде
};
```

Здесь указано, что специфический экземпляр `hash<Sales_data>` является дружественным. Поскольку данный экземпляр определяется в пространстве имен `std`, следует помнить, что этот тип хеша определяется в пространстве имен `std`. Следовательно, объявление `friend` относится к `std::hash`.



Чтобы позволить пользователям класса `Sales_data` использовать специализацию шаблона `hash`, следует определить эту специализацию в заголовке `Sales_data`.

Частичная специализация шаблона класса

В отличие от шаблона функции, специализация шаблона класса не обязана предоставлять аргументы для каждого параметра шаблона. Можно определить некоторые из них, но не все.

Частичная специализация (partial specialization) шаблона класса сама является шаблоном. Пользователи должны предоставить аргументы для тех параметров шаблона, которые не затронуты специализацией.



Частично можно специализировать только шаблон класса. Нельзя частично специализировать шаблон функции.

Библиотечный тип `remove_reference` был представлен в разделе 16.2.3 (стр. 860), он работает с серией специализаций:

```
// первоначальный, наиболее общий шаблон
template <class T> struct remove_reference {
    typedef T type;
};
// частичные специализации, которые будут использоваться для ссылок
// на l- и r-значения
template <class T> struct remove_reference<T> // ссылки на l-значение
{ typedef T type; };
template <class T> struct remove_reference<T&&> // ссылки на r-значение
{ typedef T type; };
```

Первый шаблон определяет самую общую версию. Его экземпляр может быть создан с любым типом; он использует свой аргумент шаблона как тип для своего члена `type`. Следующие два класса — это частичные специализации первоначального шаблона.

Поскольку частичная специализация — это шаблон, начнем, как обычно, с определения параметров шаблона. Подобно любой другой специализации, у частичной специализации то же имя, что и у специализируемого шаблона. Список параметров специализации шаблона включает элементы для каждого параметра шаблона, тип которого не был определен полностью при частич-

ной специализации. После имени класса располагаются аргументы для параметров специализируемого шаблона. Эти аргументы располагаются в угловых скобках после имени шаблона. Аргументы позиционально соответствуют параметрам первоначального шаблона.

Список параметров шаблона частичной специализации — это подмножество или специализация списка параметров первоначального шаблона. В данном случае у специализаций то же количество параметров, что и у первоначального шаблона. Но тип параметров в специализациях отличается от первоначального шаблона. Специализация будет использоваться для ссылок на типы l- и r-значений соответственно:

```
int i;
// decltype(42) - это int, используется первоначальный шаблон
remove_reference<decltype(42)>::type a;
// decltype(i) - это int&, используется первая (T&) частичная
// специализация
remove_reference<decltype(i)>::type b;
// decltype(std::move(i)) - это int&&, используется вторая (т.е., T&&)
// частичная специализация
remove_reference<decltype(std::move(i))>::type c;
```

У всех трех переменных, a, b и c, тип int.

Специализация членов, но не класса

Вместо специализации всего шаблона можно специализировать только одну или несколько его функций-членов. Например, если Foo — это шаблон класса с членом Bar, можно специализировать только этот член:

```
template <typename T> struct Foo {
    Foo(const T &t = T()): mem(t) { }
    void Bar() { /* ... */ }
    T mem;
    // другие члены класса Foo
};
template<> // специализация шаблона
void Foo<int>::Bar() // специализация члена Bar класса Foo<int>
{
    // осуществить всю специализированную обработку, относящуюся к целым
    // числам
}
```

Здесь специализируется только один член класса Foo<int>. Другие его члены предоставляются шаблоном Foo:

```
Foo<string> fs; // создает экземпляр Foo<string>::Foo()
fs.Bar();      // создает экземпляр Foo<string>::Bar()
Foo<int> fi;    // создает экземпляр Foo<int>::Foo()
fi.Bar();      // использует специализацию Foo<int>::Bar()
```

При использовании шаблона Foo с любым типом, кроме int, члены экземпляра создаются, как обычно. При использовании шаблона Foo с типом int все члены экземпляра, кроме Bar, создаются, как обычно. Если использовать член Bar класса Foo<int>, то получится специализированное определение.

Упражнения раздела 16.5

Упражнение 16.62. Определите собственную версию класса `hash<Sales_data>` и контейнер `unordered_multiset` объектов класса `Sales_data`. Поместите в контейнер несколько транзакций и выведите его содержимое.

Упражнение 16.63. Определите шаблон функции для подсчета количества вхождений заданного значения в векторе. Проверьте программу, передав ей вектор значений типа `double`, вектор целых чисел и вектор строк.

Упражнение 16.64. Напишите специализированную версию шаблона из предыдущего упражнения для обработки вектора `vector<const char*>` и используйте ее в программе.

Упражнение 16.65. В разделе 16.3 (стр. 877) были определены две перегруженных версии функции `debug_rep()`, одна из которых получает параметр типа `const char*`, а вторая — типа `char*`. Перепишите эти функции как специализации.

Упражнение 16.66. Каковы преимущества и недостатки перегрузки функций `debug_rep()` по сравнению с определением специализаций?

Упражнение 16.67. Повлияет ли определение этих специализаций на подбор функций `debug_rep()`? Почему?

Резюме

Шаблоны — это отличительная особенность языка C++ и основа его стандартной библиотеки. Шаблон представляет собой независимый от типа “чертеж”, используемый компилятором для создания конкретных экземпляров указанных классов или функций. Шаблон разрабатывается один раз, а его экземпляры компилятор создает для соответствующего типа или значения по мере его применения.

Можно определять шаблоны функций и классов. Библиотечные алгоритмы являются шаблонами функций, а библиотечные контейнеры — шаблонами классов.

Явный аргумент шаблона позволяет фиксировать тип или значение одного или нескольких параметров шаблона. К параметрам с явным аргументом шаблона применимы нормальные преобразования.

Специализация шаблона — это отдельное специальное определение, позволяющее создать такую версию шаблона, в которой для одного или нескольких параметров указан определенный тип или значение. Специализация полезна в случае, когда для некоторых типов стандартное определение шаблона неприменимо.

Главная часть последнего выпуска стандарта языка C++ относится к шаблонам с переменным количеством аргументов. Такой шаблон способен получать переменное количество аргументов разных типов. Шаблоны с переменным количеством аргументов позволяют написать такие функции, как функция-член `emplace()` классов контейнеров и библиотечная функция `make_shared()`, передающая аргументы конструктору объекта.

Термины

Аргумент шаблона (template argument). Тип или значение, указанные при создании экземпляра шаблона.

Аргумент шаблона по умолчанию (default template argument). Тип или значение, используемые при создании экземпляра шаблона, если пользователь не предоставил соответствующий аргумент.

Дедукция аргумента шаблона (template argument deduction). Процесс, в ходе которого компилятор выясняет, какой экземпляр шаблона функции следует создать. Для этого компилятор исследует типы аргументов, переданных в качестве параметров шаблона. На основании полученных типов или значений объектов, связанных с параметрами шаблона, компилятор автоматически создает соответствующую версию функции.

Пакет параметров (parameter pack). Параметр шаблона или функции, представляющий любое количество параметров.

Пакет параметров функции (function parameter pack). Пакет, представляющий любое количество параметров функций.

Пакет параметров шаблона (template parameter pack). Пакет, представляющий любое количество параметров шаблона.

Параметр значения (nontype parameter). Параметр шаблона, представляющий значение. Во время создания экземпляра шаблона класса каждый параметр значения связывается с константным выражением, переданным в качестве аргумента при создании экземпляра класса.

Параметр типа (type parameter). Имя, используемое в списке параметров шаблона вместо имени типа. Параметры типа определяются после ключевого слова `typename` или `class`.

Параметр шаблона (template parameter). Имя, определенное в списке параметров шаблона и используемое в определении его экземпляров. Параметр шаблона может быть типом или значением. Чтобы использовать шаблон класса, следует предоставить явные аргументы для каждого параметра шаблона. Компилятор использует эти типы или значения при создании версии экземпляра класса. При этом используемые параметры заменяются фактическими аргументами. Когда используется шаблон функции, компилятор выводит аргументы шаблона из аргументов вызова и создает экземпляр специфической функции на их основании.

Развертывание пакета (pack expansion). Процесс, в ходе которого пакет параметров заменяется соответствующим списком его элементов.

Создание экземпляра (instantiate). Процесс компилятора, в ходе которого соответствующие параметры шаблона заменяются фактическими аргументами и создается

специфический экземпляр шаблона. Экземпляры функций создаются автоматически на основании аргументов, использованных в вызове. При использовании шаблона класса следует явно предоставить аргументы шаблона.

Создание экземпляра (instantiation). Процесс создания компилятором класса или функции из шаблона.

Специализация шаблона (template specialization). Переопределение всего шаблона класса, или его члена, или шаблона функции, в котором определены параметры шаблона. Специализация шаблона не может быть осуществлена до завершения определения шаблона класса, подвергающегося специализации. Специализация шаблона должна быть осуществлена прежде, чем он будет использован для специализированных аргументов. Каждый параметр шаблона в шаблоне функции должен быть специализирован полностью.

Список параметров шаблона (template parameter list). Список параметров типа или значения (разделяемый запятыми), используемый в определении или объявлении шаблона.

Схема (pattern). Определяет форму каждого элемента в развернутом пакете параметров.

Трансформация типа (type transformation). Определенные библиотекой шаблоны класса, преобразующие предоставленный параметр типа шаблона в связанный тип.

Частичная специализация (partial specialization). Версия шаблона класса, в которой определены некоторые, но не все параметры шаблона либо некоторые параметры определены не полностью.

Шаблон класса (class template). Определение, которое может быть использовано при создании экземпляров специфических классов. При определении шаблона класса используется ключевое слово `template`, за которым следует разделяемый запятыми список параметров, заключенный в угловые скобки (`<>`).

Шаблон с переменным количеством аргументов (variadic template). Шаблон, получающий переменное количество аргументов. Пакет параметров шаблона определяется с использованием многоточия (например, `class...`, `typename...` или `имя_типа...`).

Шаблон функции (function template). Определение, которое может быть использовано при создании экземпляра специфической функции. При определении шаблона функции используется ключевое слово `template`, за которым следует разделяемый запятыми список параметров, заключенный в угловые скобки (`<>`), и определение функции.

Шаблон-член (member template). Член класса или шаблона класса, который является шаблоном функции. Шаблон-член не может быть виртуальным.

Явное создание экземпляра (explicit instantiation). Объявление, предоставляющее явные аргументы для всех параметров шаблона. Используется для управления процессом создания экземпляра. Если объявление будет внешним (`extern`), то экземпляр шаблона не будет создан; в противном случае создается экземпляр шаблона с указанными аргументами. Для каждого внешнего объявления шаблона где-нибудь в программе должно быть внутреннее явное создание экземпляра.

Явный аргумент шаблона (explicit template argument). Аргумент шаблона, предоставляемый пользователем при вызове функции или определении типа шаблона класса. Явные аргументы шаблона указывают в угловых скобках непосредственно после имени шаблона.